LAPORAN TUGAS BESAR 1
IF3270 PEMBELAJARAN MESIN
Feedforward Neural Network

Disusun oleh:

Kelompok 9 ciamso

Anella Utari Gunadi	13523078
Nayla Zahira	13523079
Diyah Susan Nugrahani	13523080

SEKOLAH TINGGI ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI 1
DAFTAR GAMBAR 2
BAB 1 DESKRIPSI PERSOALAN 3
BAB 2 PEMBAHASAN 4
2.1 Penjelasan Implementasi 4
2.1.1 Deskripsi Kelas 4
2.1.1.1 Kelas Layer 4
2.1.1.2 Kelas DenseLayer 5
2.1.1.3 Kelas RMSNormLayer 8
2.1.1.4 Kelas ActivationLayer 10
2.1.1.5 Kelas ActivationFunctions 11
2.1.1.6 Kelas LossFunctions 13
2.1.1.7 Kelas Initializers 14
2.1.1.8 Kelas NeuralNetwork 16
2.1.1.9 Kelas FFNN 20
2.1.2 Forward Propagation 23
2.1.3 Backward Propagation dan Weight Update 23
2.1.3.1 Backward Propagation 23
2.1.3.2 Weight Update 24
2.2 Hasil Pengujian 24
2.2.1 Pengujian Hyperparameter 24
2.2.1.1 Pengaruh Depth dan Width 24
2.2.1.2 Pengaruh Fungsi Aktivasi Hidden Layer 26
2.2.1.3 Pengaruh Learning Rate 30
2.2.1.4 Pengaruh Regularisasi 33
2.2.1.5 Pengaruh Normalisasi RMSNorm 37
2.2.1.6 Uji Perbandingan Library Sklearn 39
2.2.1.7 Pengujian Optimizer Adaptive Moment Estimation (Adam) 41
BAB 3 KESIMPULAN DAN SARAN 43
3.1 Kesimpulan 43
3.2 Saran 43
PEMBAGIAN TUGAS 44
REFERENSI 45
LAMPIRAN 46

DAFTAR GAMBAR

Gambar 2.2.1.1.1 Grafik Pengujian Depth..... 24
Gambar 2.2.1.1.2 Grafik Pengujian Width..... 25
Gambar 2.2.1.2.1 Perbandingan Evaluasi Model Berdasarkan Variasi Fungsi Aktivasi.....26
Gambar 2.2.1.3.4 Grafik Weight Distribution pada Model dengan Fungsi Aktivasi ReLU..... 28
Gambar 2.2.1.3.5 Grafik Weight Distribution pada Model dengan Fungsi Aktivasi Sigmoid.....28
Gambar 2.2.1.3.7 Grafik Weight Distribution pada Model dengan Fungsi Aktivasi Leaky ReLU..... 28
Gambar 2.2.1.3.8 Grafik Weight Distribution pada Model dengan Fungsi Aktivasi ELU..... 29
Gambar 2.2.1.3.9 Grafik Weight Distribution pada Model dengan Fungsi Aktivasi Swish..... 29
Gambar 2.2.1.3.10 Grafik Gradient Distribution pada Model dengan Fungsi Aktivasi Linear..... 29
Gambar 2.2.1.3.11 Grafik Gradient Distribution pada Model dengan Fungsi Aktivasi ReLU.....29
Gambar 2.2.1.3.12 Grafik Gradient Distribution pada Model dengan Fungsi Aktivasi Sigmoid..... 29
Gambar 2.2.1.3.13 Grafik Gradient Distribution pada Model dengan Fungsi Aktivasi Tanh.....30
Gambar 2.2.1.3.14 Grafik Gradient Distribution pada Model dengan Fungsi Aktivasi Leaky ReLU..30
Gambar 2.2.1.3.15 Grafik Gradient Distribution pada Model dengan Fungsi Aktivasi ELU.....30
Gambar 2.2.1.3.16 Grafik Gradient Distribution pada Model dengan Fungsi Aktivasi Swish.....30
Gambar 2.2.1.3.1 Perbandingan Evaluasi Model Berdasarkan Variasi Learning Rate..... 31
Gambar 2.2.1.3.2 Grafik Training dan Validation Loss.....31
Gambar 2.2.1.3.3 Grafik Weight Distribution pada Model dengan Learning Rate 0,001.....32
Gambar 2.2.1.3.5 Grafik Weight Distribution pada Model dengan Learning Rate 0,1.....32
Gambar 2.2.1.3.6 Grafik Gradient Distribution pada Model dengan Learning Rate 0,001..... 33
Gambar 2.2.1.3.7 Grafik Gradient Distribution pada Model dengan Learning Rate 0,01..... 33
Gambar 2.2.1.3.7 Grafik Gradient Distribution pada Model dengan Learning Rate 0,1..... 33
Gambar 2.2.1.4.1 Grafik Training Loss dan Validation Loss Uji Regularisasi..... 34
Gambar 2.2.1.4.2 Grafik Weight Distribution pada Model Tanpa Regularisasi.....34
Gambar 2.2.1.4.3 Grafik Weight Distribution pada Model Regularisasi L1.....35
Gambar 2.2.1.4.4 Grafik Weight Distribution pada Model Regularisasi L2.....35
Gambar 2.2.1.4.5 Weight Gradient Layer 0 pada Model Tanpa Regularisasi..... 36
Gambar 2.2.1.4.6 Weight Gradient Layer 0 pada Model Regularisasi L1..... 36
Gambar 2.2.1.4.7 Weight Gradient Layer 0 pada Model Regularisasi L2..... 36
Gambar 2.2.1.5.1 Grafik Training Loss dan Validation Loss Uji RMSNorm..... 37
Gambar 2.2.1.5.2 Grafik Weight Distribution Model Tanpa Normalisasi.....38
Gambar 2.2.1.5.3 Grafik Weight Distribution Model dengan RMSNorm..... 38
Gambar 2.2.1.5.4 Grafik Weight Gradient Distribution Layer 0 pada Model Tanpa Normalisasi.....39
Gambar 2.2.1.5.4 Grafik Weight Gradient Distribution Layer 0 pada Model dengan RMSNorm..... 39
Gambar 2.2.1.6.1 Hasil Evaluasi Model FFNN dan sklearn..... 40
Gambar 2.2.1.6.2 Tabel Perbandingan Nilai Accuracy dan Macro F1 FFNN dan sklearn..... 40
Gambar 2.2.1.7.1 Grafik Training Loss dan Validation Loss pada GD vs Adam..... 41
Gambar 2.2.1.7.2 Perbandingan Hasil Prediksi GD dan Adam..... 41

BAB 1 DESKRIPSI PERSOALAN

Pada Tugas Besar 1 IF3270 Pembelajaran Mesin ini, persoalan utama yang harus diselesaikan adalah mengimplementasikan *Feedforward Neural Network* (FFNN) *from scratch* dengan menggunakan bahasa python dan memanfaatkan library numpy. Implementasi dilakukan dengan pendekatan modular, sehingga model dapat menerima variasi jumlah layer dan neuron, serta mendukung berbagai fungsi aktivasi seperti linear, ReLU, sigmoid, tanh, dan softmax, serta beberapa fungsi loss seperti MSE, Binary Cross-Entropy, dan Categorical Cross-Entropy.

Selain itu, model juga harus memiliki mekanisme inisialisasi bobot yang beragam, mendukung proses *forward propagation* dan *backward propagation* menggunakan *chain rule*, serta melakukan pembaruan bobot menggunakan metode *gradient descent*. Model juga dilengkapi dengan kemampuan menyimpan dan menampilkan distribusi bobot serta gradien, serta mendukung proses *training* berbasis *batch* dengan parameter seperti *learning rate, batch size, epoch,* dan *verbose*.

Setelah selesai diimplementasikan, dilakukan berbagai eksperimen untuk menganalisis pengaruh hyperparameter seperti depth, width, fungsi aktivasi, dan learning rate, serta pengaruh regularisasi L1 dan L2 terhadap performa model. Selain itu, hasil prediksi model juga dibandingkan dengan model dari library sklearn untuk melihat perbedaan performa antara implementasi manual dan model yang telah dioptimasi.

Pada tugas ini digunakan dataset Global Student Placement & Salary Dataset, yang berisi informasi mengenai performa akademik dan faktor-faktor lain yang mempengaruhi penempatan kerja mahasiswa. Target yang diprediksi adalah *placement_status*.

BAB 2 PEMBAHASAN

2.1 Penjelasan Implementasi

2.1.1 Deskripsi Kelas

Pada tugas besar ini, kami menggunakan prinsip pemrograman berorientasi objek agar lebih modular. Berikut merupakan penjelasan lebih detail dari masing-masing kelas yang digunakan.

2.1.1.1 Kelas *Layer*

Dalam mengimplementasikan *Feedforward Neural Network* dari dasar. Struktur utama dibangun di atas sebuah *base class* abstrak yang bernama Layer. Kelas ini berfungsi sebagai kerangka dasar yang mendefinisikan standar komunikasi antar lapisan menggunakan dua metode utama yaitu *forward* untuk memproses data dari input menuju output dan *backward* untuk menghitung galat error dari output kembali ke input. Dengan menggunakan kelas abstrak, setiap komponen dapat dikembangkan dengan cara menurunkan kelas tersebut tanpa mengubah logika utama.

Base Class Layer

```
class Layer:
    def __init__(self):
        self.input = None
        self.output = None

    def forward(self, input_data):
        raise NotImplementedError

    def backward(self, output_error, learning_rate):
        raise NotImplementedError
```

Nama *Method*	Penjelasan
forward(self, input_data)	*Method forward* yang wajib diimplementasikan oleh kelas turunannya.
backward(self, output_error, learning_rate)	*Method backward* yang wajib diimplementasikan oleh kelas turunannya.

2.1.1.2 Kelas *DenseLayer*

Kelas yang diturunkan dari kelas *layer* adalah DenseLayer. Kelas ini bertanggung jawab atas proses pembelajaran parameter di dalam model. DenseLayer melakukan transformasi linear terhadap data input dengan mengalikan matriks input terhadap bobot dan menambahkannya dengan bias. Secara matematis, dapat dirumuskan sebagai $Z = XW + b$. Selama fase *training, layer* ini akan menyimpan status input yang kemudian digunakan pada fase *backpropagation.* Pada tahap tersebut, DenseLayer akan menghitung gradien terhadap bobot dan bias menggunakan aturan rantai lalu memperbarui nilai-nilai tersebut menggunakan *learning rate* untuk meminimalkan *loss function.*

DenseLayer

```
class DenseLayer(Layer):
    def __init__(self, input_size, output_size, initializer_func,
l1_lambda=0.0, l2_lambda=0.0, **kwargs):
        self.weights = initializer_func((input_size, output_size),
**kwargs)
        self.bias = np.zeros((1, output_size)) # biasanya nol di awal,
cmiiw

        # regularization parameters
        self.l1_lambda = l1_lambda
        self.l2_lambda = l2_lambda

        # dipake buat backward propagation
        self.weights_gradient = None
        self.bias_gradient = None
        self.input = None

        # state utk adam
        self.m_w = np.zeros_like(self.weights)
        self.v_w = np.zeros_like(self.weights)
        self.m_b = np.zeros_like(self.bias)
        self.v_b = np.zeros_like(self.bias)

    def forward(self, input_data):
        # ngitung sigma atau Z = XW + b
        self.input = input_data
        return np.dot(self.input, self.weights) + self.bias

    def backward(self, output_error, learning_rate, optimizer="gd", t=1,
beta1=0.9, beta2=0.999, epsilon=1e-8):

        # menghitung gradient menggunakan chain rule untuk memperbarui
bobot
        input_error = np.dot(output_error, self.weights.T)
        batch_size = self.input.shape[0]

        self.weights_gradient = np.dot(self.input.T, output_error) /
batch_size
        self.bias_gradient = np.sum(output_error, axis=0, keepdims=True) /
batch_size
```

```
        if self.l1_lambda > 0:
            self.weights_gradient += self.l1_lambda * np.sign(self.weights)
/ batch_size

        if self.l2_lambda > 0:
            self.weights_gradient += self.l2_lambda * self.weights /
batch_size

        if optimizer == "gd":
            self.weights -= learning_rate * self.weights_gradient
            self.bias -= learning_rate * self.bias_gradient

        elif optimizer == "adam":
            # momentum pertama
            self.m_w = beta1 * self.m_w + (1 - beta1) *
self.weights_gradient
            self.m_b = beta1 * self.m_b + (1 - beta1) * self.bias_gradient

            # momentum kedua
            self.v_w = beta2 * self.v_w + (1 - beta2) *
(self.weights_gradient ** 2)
            self.v_b = beta2 * self.v_b + (1 - beta2) * (self.bias_gradient
** 2)

            # bias correction
            m_w_hat = self.m_w / (1 - beta1 ** t)
            m_b_hat = self.m_b / (1 - beta1 ** t)
            v_w_hat = self.v_w / (1 - beta2 ** t)
            v_b_hat = self.v_b / (1 - beta2 ** t)

            self.weights -= learning_rate * m_w_hat / (np.sqrt(v_w_hat) +
epsilon)
            self.bias -= learning_rate * m_b_hat / (np.sqrt(v_b_hat) +
epsilon)

        else:
            raise ValueError(f"Optimizer '{optimizer}' tidak dikenali.")

        return input_error

    def get_regularization_loss(self):
        reg_loss = 0.0
        if self.l1_lambda > 0:
            reg_loss += self.l1_lambda * np.sum(np.abs(self.weights))
        if self.l2_lambda > 0:
            reg_loss += 0.5 * self.l2_lambda *
np.sum(np.square(self.weights))
        return reg_loss
```

Atribut	Penjelasan
weights	Matriks bobot yang menghubungkan setiap

	neuron input ke neuron output
bias	Vektor tambahan untuk setiap neuron output guna menggeser fungsi aktivasi agar lebih fleksibel dalam mencocokan data
input	Tempat penyimpanan sementara data input saat *forward pass*
l1_lambda	Koefisien untuk L1 Regularization (Lasso). Berfungsi mendorong bobot yang kurang penting menjadi tepat nol sehingga model lebih simpel
l2_lambda	Koefisien untuk L2 Regularization (Ridge). Berfungsi mencegah bobot tumbuh terlalu besar
weights_gradient	Nilai perubahan yang diperlukan untuk bobot berdasarkan error yang ditemukan
bias_gradient	Nilai perubahan yang diperlukan untuk bias berdasarkan error yang ditemukan
m_w	Momentum orde pertama (*mean*) untuk bobot pada *optimizer* Adam
v_w	Momentum orde kedua (*variance*) untuk bobot pada *optimizer* Adam
m_b	Momentum orde pertama untuk bias
v_b	Momentum orde kedua untuk bias

Method	Penjelasan
__init__(input_size, output_size, initializer_func, l1_lambda, l2_lambda)	Inisialisasi kelas. Menyiapkan struktur layer, menentukan ukuran input-output, dan memanggil fungsi inisialisasi bobot dan mengatur regularisasi.
forward(input_data)	Melakukan operasi linear matriks dengan rumus $Z = XW + b$. Hasilnya diteruskan ke lapisan aktivasi.
backward(output_error, learning_rate,	Menggunakan *chain rule* untuk menghitung

optimizer, t, beta1, beta2, epsilon)	gradien dan *update* bobot berdasarkan optimizer yang digunakan. Jika menggunakan Gradient Descent, bobot diperbarui langsung menggunakan *learning rate.* Jika menggunakan Adam, *update* dilakukan menggunakan momentum orde pertama dan kedua.

2.1.1.3 Kelas RMSNormLayer

Kelas RMSNormLayer adalah layer tambahan yang digunakan untuk melakukan normalisasi terhadap *output* dari layer sebelumnya, RMSNorm bekerja dengan menormalkan input berdasarkan akar rata-rata kuadrat nilai input tanpa mengurangi nilai *mean*. Berbeda dengan *batch normalization*, RMSNorm tidak bergantung pada distribusi *batch*, sehingga lebih sederhana dan stabil dalam implementasi. Layer ini memiliki parameter *trainable* berupa *gamma* yang berfungsi untuk melakukan *scaling* terhadap normalisasi. Selama proses *forward*, layer akan menghitung nilai RMS dari input, kemudian melakukan normalisasi dan *scaling*. Pada proses *backward*, layer akan menghitung gradien terhadap *gamma* serta meneruskan gradien ke layer sebelumnya.

RMSNormLayer

```
class RMSNormLayer(Layer):
   def __init__(self, feature_size, eps=1e-8):
        super().__init__()
        self.feature_size = feature_size
        self.eps = eps

        # parameter trainable
        self.gamma = np.ones((1, feature_size))
        self.gamma_gradient = None

        # cache forward
        self.input = None
        self.rms = None
        self.normalized = None

    def forward(self, input_data):
        self.input = input_data

        mean_square = np.mean(np.square(input_data), axis=1, keepdims=True)
        self.rms = np.sqrt(mean_square + self.eps)
        self.normalized = input_data / self.rms
        self.output = self.gamma * self.normalized

        return self.output

    def backward(self, output_error, learning_rate, optimizer="gd", t=1,
```

```
beta1=0.9, beta2=0.999, epsilon=1e-8):
        batch_size = self.input.shape[0]
        d = self.input.shape[1]

        self.gamma_gradient = np.sum(output_error * self.normalized,
axis=0, keepdims=True) / batch_size
        dxhat = output_error * self.gamma
        dot = np.sum(dxhat * self.input, axis=1, keepdims=True)
        input_error = (dxhat / self.rms) - (self.input * dot) / (d *
(self.rms ** 3))

        # update gamma
        self.gamma -= learning_rate * self.gamma_gradient

        return input_error
```

Atribut	Penjelasan
feature_size	Jumlah fitur pada input yang akan dinormalisasi
eps	Nilai kecil untuk menghindari pembagian dengan nol
gamma	Parameter *trainable* untuk *scaling* hasil normalisasi
gamma_gradient	Gradien dari parameter gamma
input	Menyimpan input dari *forward pass*
rms	Nilai *root mean square* dari input
normalized	Hasil normalisasi input

Method	Penjelasan
__init__(feature_size, eps)	Menginisialisasi layer dengan ukuran fitur dan parameter epsilon.
forward(input_data)	Menghitung RMS dari input, melakukan normalisasi, dan mengalikan dengan parameter gamma.
backward(output_error, learning_rate, optimizer, t, beta1, beta2, epsilon)	Menghitung gradien terhadap gamma dan input, serta melakukan *update* parameter gamma menggunakan *learning rate*.

2.1.1.4 Kelas *ActivationLayer*

Kelas *ActivationLayer* berperan dalam memberikan sifat non-linieritas pada model. *Layer* ini bekerja dengan cara membungkus hasil perhitungan pada *DenseLayer* dan menerapkannya ke dalam fungsi aktivasi tertentu seperti fungsi aktivasi tanh, sigmoid, dan lainnya. Dalam proses *backward, layer* ini berfungsi untuk menghitung turunan dari fungsi aktivasi tersebut untuk meneruskan gradien error ke lapisan di bawahnya.

ActivationLayer

```
class ActivationLayer(Layer):
    def __init__(self, activation, activation_prime):
        super().__init__()
        self.activation = activation
        self.activation_prime = activation_prime

    def forward(self, input_data):
        self.input = input_data # simpan untuk backward pass
        self.output = self.activation(self.input) # terapkan fungsi
aktivasi
        return self.output

    def backward(self, output_error, learning_rate):
        return self.activation_prime(self.input) * output_error
```

Atribut	Penjelasan
activation	Menyimpan fungsi matematika non-linear yang akan digunakan sebagai fungsi aktivasi
activation_prime	Menyimpan fungsi turunan dari fungsi aktivasi. Digunakan untuk menghitung gradien saat proses *backward.*
input	Menyimpan data sli yang masuk ke layer ini sebelum diolah. Data ini digunakan untuk menghitung turunan pada fase *backward.*
output	Hasil akhir setelah input dimasukkan ke dalam fungsi aktivasi.

Method	Penjelasan
__init__	Menerima dua fungsi utama, fungsi aktivasi

	dan turunannya, lalu menyimpannya ke dalam atribut objek.
forward	Menerapkan fungsi aktivasi pada input data. Nilai dari *DenseLayer* akan diolah sesuai dengan fungsi aktivasinya menjadi output
backward	Menghitung seberapa besar error yang harus diteruskan ke layer sebelumnya dengan rumus turunan fungsi aktivasi dikali error dari output

2.1.1.5 Kelas *ActivationFunctions*

Kelas *ActivationFunction* berisi formula untuk fungsi aktivasi beserta dengan turunan pertamanya. Terdapat lima fungsi aktivasi yang kami implementasikan pada tugas besar ini, yaitu linear, relu, sigmoid, tanh, dan softmax. Atribut dari kelas ini hanya satu yaitu x sebagai nilai input. Keluaran dari kelas ini berupa nilai konstan.

ActivationFunctions

```
class ActivationFunctions:
    def __init__(self):
        pass

    def linear(self, x):
        return x

    def linear_prime(self, x):
        return np.ones_like(x)

    def relu(self, x):
        return np.maximum(0, x)

    def relu_prime(self, x):
        return (x > 0).astype(float)

    def sigmoid(self, x):
        return 1/(1+ np.exp(-x))

    def sigmoid_prime(self, x):
        return self.sigmoid(x) * (1 - self.sigmoid(x))

    def hyperbolic_tangent(self, x):
        return np.tanh(x)

    def hyperbolic_tangent_prime(self, x):
        return 1 - np.tanh(x)**2

    def softmax(self, x):
```

```
        x = np.array(x)
        x_shifted = x - np.max(x, axis=1, keepdims=True)
        exp_x = np.exp(x_shifted)
        return exp_x / np.sum(exp_x, axis=1, keepdims=True)

    def softmax_prime(self, x):
        s = self.softmax(x)
        batch_jacobian = []

        for row in s:
            row = row.reshape(-1, 1)
            jacobian = np.diagflat(row) - np.dot(row, row.T)
            batch_jacobian.append(jacobian)

        return np.array(batch_jacobian)

    def leaky_relu(self, x, alpha=0.01):
        return np.where(x > 0, x, alpha * x)

    def leaky_relu_prime(self, x, alpha=0.01):
        dx = np.ones_like(x)
        dx[x < 0] = alpha
        return dx

    def elu(self, x, alpha=1.0):
        return np.where(x > 0, x, alpha * (np.exp(x) - 1))

    def elu_prime(self, x, alpha=1.0):
        return np.where(x > 0, 1, alpha * np.exp(x))

    def swish(self, x, b=1.0):
        return x * self.sigmoid(b * x)

    def swish_prime(self, x, b=1.0):
        sigmoid_bx = self.sigmoid(b * x)
        return sigmoid_bx + b * x * sigmoid_bx * (1 - sigmoid_bx)
```

Nama *Method*	Penjelasan
linear(self, x):	Fungsi identitas dimana output = input
linear_prime(self, x)	Turunan pertama dari fungsi linear yang bernilai konstan 1
relu(self, x)	Fungsi ReLU yang mengubah nilai negatif menjadi nol.
relu_prime(self, x)	Turunan pertama dari fungsi ReLU, jika x > 0 return 1, jika tidak maka return 0.
sigmoid(self, x)	Fungsi sigmoid yang mengubah input ke

	rentang 0 hingga 1.
sigmoid_prime(self, x)	Turunan pertama dari fungsi sigmoid.
hyperbolic_tangent(self, x)	Fungsi tanh yang outputnya berada pada rentang -1 hingga 1.
hyperbolic_tangent_prime(self, x)	Turunan pertama dari fungsi tanh.
softmax(self, x)	Fungsi aktivasi untuk klasifikasi multiclass dengan output berbentuk distribusi probabilitas.
softmax_prime(self, x)	Turunan pertama dari fungsi softmax.

2.1.1.6 Kelas *LossFunctions*

LossFunction

```
class LossFunctions:
    def __init__(self):
        pass

    def mse(self, y_true, y_pred):
        return np.mean(np.power(y_true - y_pred, 2))

    def mse_prime(self, y_true, y_pred):
        return 2 * (y_pred - y_true)

    def binary_cross_entropy(self, y_true, y_pred):
        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1. - epsilon)
        return -np.mean(y_true * np.log(y_pred) + (1-y_true) *
np.log(1-y_pred))

    def bce_prime(self, y_true, y_pred):
        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1. - epsilon)
        n = y_true.size

        return -(y_true / y_pred - (1 - y_true) / (1 - y_pred))

    def categorical_cross_entropy(self, y_true, y_pred):
        n = y_true.shape[0]
        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1. - epsilon)
        return -np.sum(y_true * np.log(y_pred)) / n

    def cce_prime(self, y_true, y_pred):
        n = y_true.shape[0]
        epsilon = 1e-15
```

```
        y_pred = np.clip(y_pred, epsilon, 1. - epsilon)
        return -(y_true / y_pred)
```

Kelas *LossFunctions* merupakan matriks evaluasi yang bertugas untuk mengukur sejauh mana ketidakakuratan prediksi yang dihasilkan oleh model jika dibandingkan dengan nilai target sebenarnya. Besarnya nilai yang dihasilkan oleh fungsi loss mencerminkan kualitas dari parameter yang dimiliki model saat ini. Semakin tinggi nilai loss, berarti model melakukan kesalahan prediksi yang semakin besar, sedangkan jika nilai loss mendekati nol maa prediksi model sudah bagus. Gradien dari nilai loss ini yang menentukan ke arah mana bobot harus diubah agar meminimalkan kesalahan pada iterasi berikutnya.

Nama *Method*	Penjelasan
mse(self, y_true, y_pred)	Mean Squared Error untuk regresi
mse_prime(self, y_true, y_pred)	Turunan pertama MSE
binary_cross_entropy(self, y_true, y_pred)	Loss untuk klasifikasi biner
bce_prime(self, y_true, y_pred)	Turunan pertama dari cross-entropy
categorical_cross_entropy(self, y_true, y_pred)	Digunakan untuk klasifikasi multi kelas
cce_prime(self, y_true, y_pred)	Turunan dari categorical cross-entropy

2.1.1.7 Kelas *Initializers*

Kelas *Initializers* merupakan proses menentukan nilai awal bagi parameter internal model, bobot dan bias, sebelum proses pelatihan dimulai. Dalam tugas besar ini, kami mengimplementasikan lima inisialisasi bobot.

Initializers

```
class Initializers:
    def __init__(self):
        pass

    def zero_init(self, shape):
        # isi dengan nol
        return np.zeros(shape)

    def uniform_init(self, shape, low, high, seed=None):
        # random dengan distribusi uniform
        if seed is not None:
```

```
            np.random.seed(seed)
        return np.random.uniform(low, high, size=shape)

    def normal_init(self, shape, mean, variance, seed=None):
        # random dengan distribusi normal
        if seed is not None:
            np.random.seed(seed)

        # standar deviasi
        std_dev = np.sqrt(variance)
        return np.random.normal(mean, std_dev, size=shape)

    def xavier_init(self, shape, seed=None):
        if seed is not None:
            np.random.seed(seed)

        n_in, n_out = shape
        limit = np.sqrt(6.0 / (n_in + n_out))
        return np.random.uniform(-limit, limit, size=shape)

    def he_init(self, shape, seed=None):
        if seed is not None:
            np.random.seed(seed)

        n_in, n_out = shape
        std = np.sqrt(2.0 / n_in)
        return np.random.normal(0, std, size=shape)
```

Nama *Method*	Penjelasan
zero_init(self, shape)	Inisialisasi bobot dengan semua bernilai nol.
uniform_init(self, shape, low, high, seed=None)	Inisialisasi bobot dengan nilai acak dari distribusi uniform.
normal_init(self, shape, mean, variance, seed=None)	Inisialisasi bobot dengan nilai acak dari distribusi normal.
xavier_init(self, shape, seed=None)	Inisialisasi yang cocok untuk aktivasi sigmoid atau tanh.
he_init(self, shape, seed=None)	Inisialisasi yang cocok untuk aktivasi ReLU.

2.1.1.8 Kelas *NeuralNetwork*

Kelas *NeuralNetwork* merupakan kelas utama yang digunakan untuk membangun dan menjalani model FeedForward Neural Network. Kelas ini berfungsi untuk menyimpan kumpulan *layer* yang membentuk jaringan serta mengatur proses pelatihan model. Di dalam

kelas ini terdapat mekanisme untuk menambahkan *layer*, menentukan fungsi *loss*, melakukan proses training menggunakan *forward propagation* dan *backward propagation*, serta melakukan prediksi terhadap data baru. Selain itu, kelas ini juga menyediakan fungsi untuk menyimpan dan memuat model serta menampilkan distribusi bobot dan gradien pada layer tertentu untuk keperluan analisis.

NeuralNetwork

```
class NeuralNetwork:
    def __init__(self):
        self.layers = []
        self.history = {'train_loss': [], 'val_loss': []}
        self.loss = None
        self.loss_prime = None

    def use(self, loss, loss_prime):
        self.loss = loss
        self.loss_prime = loss_prime

    def save(self, filepath):
        with open(filepath, 'wb') as f:
            pickle.dump(self, f)

    def load(self, filepath):
        with open(filepath, 'rb') as f:
            loaded_model = pickle.load(f)
        self.layers = loaded_model.layers
        self.history = loaded_model.history
        self.loss = loaded_model.loss
        self.loss_prime = loaded_model.loss_prime

    def add(self, layer):
        self.layers.append(layer)

    def get_total_regularization_loss(self):
        reg_loss = 0.0
        for layer in self.layers:
            if hasattr(layer, 'get_regularization_loss'):
                reg_loss += layer.get_regularization_loss()
        return reg_loss

    def predict(self, input_data):
        output = input_data.copy()
        for layer in self.layers:
            output = layer.forward(output)
        return output

    def train(self, x_train, y_train, x_val=None, y_val=None, epochs=10,
batch_size=32, learning_rate=0.01, verbose=1, optimizer="gd", beta1=0.9,
beta2=0.999, epsilon=1e-8):
        self.history = {'train_loss': [], 'val_loss': []}

        n_samples = len(x_train)
        n_batches = int(np.ceil(n_samples / batch_size))
```

```
        t = 0 # timestamp adam

        epoch_logs = []

        for epoch in range(epochs):
            indices = np.random.permutation(n_samples)
            x_train_shuffled = x_train[indices]
            y_train_shuffled = y_train[indices]

            train_loss = 0.0

            for batch_idx in range(n_batches):
                start_idx = batch_idx * batch_size
                end_idx = min(start_idx + batch_size, n_samples)

                x_batch = x_train_shuffled[start_idx:end_idx]
                y_batch = y_train_shuffled[start_idx:end_idx]

                # forward
                output = x_batch
                for layer in self.layers:
                    output = layer.forward(output)

                # data loss
                batch_data_loss = self.loss(y_batch, output)

                # backward + update
                error = self.loss_prime(y_batch, output)
                t += 1
                for layer in reversed(self.layers):
                    error = layer.backward(error, learning_rate,
optimizer=optimizer, t=t, beta1=beta1, beta2=beta2, epsilon=epsilon)

                batch_reg_loss = self.get_total_regularization_loss()
                batch_total_loss = batch_data_loss + batch_reg_loss

                train_loss += batch_total_loss * len(x_batch)

            train_loss /= n_samples
            self.history['train_loss'].append(train_loss)

            val_loss = None
            if x_val is not None and y_val is not None:
                val_output = x_val.copy()
                for layer in self.layers:
                    val_output = layer.forward(val_output)

                val_data_loss = self.loss(y_val, val_output)
                val_reg_loss = self.get_total_regularization_loss()
                val_loss = val_data_loss + val_reg_loss
                self.history['val_loss'].append(val_loss)

            msg = f"Epoch {epoch+1}/{epochs} | Train Loss:
{train_loss:.6f}"
            if val_loss is not None:
                msg += f" | Val Loss: {val_loss:.6f}"
```

```
            epoch_logs.append(msg)

            # progress bar
            if verbose == 1:
                progress = (epoch + 1) / epochs
                bar_length = 20
                filled = int(bar_length * progress)
                bar = "#" * filled + "-" * (bar_length - filled)

                clear_output(wait=True)
                print(f"[{bar}] {epoch+1}/{epochs}")
                print(msg)

        if verbose == 1:
            clear_output(wait=True)
            print("Training selesai.\n")
            for log in epoch_logs:
                print(log)

        return self.history

    def plot_weight_distribution(self, layer_indices):
        for i in layer_indices:
            if i < 0 or i >= len(self.layers):
                print(f"Layer index {i} tidak valid.")
                continue

            layer = self.layers[i]
            if hasattr(layer, 'weights') and layer.weights is not None:
                plt.figure(figsize=(6, 4))
                plt.hist(layer.weights.flatten(), bins=30)
                plt.title(f"Weight Distribution - Layer {i}")
                plt.xlabel("Weight value")
                plt.ylabel("Frequency")
                plt.tight_layout()
                plt.show()
            else:
                print(f"Layer {i} tidak punya weights.")

    def plot_gradient_distribution(self, layer_indices):
        for i in layer_indices:
            if i < 0 or i >= len(self.layers):
                print(f"Layer index {i} tidak valid.")
                continue

            layer = self.layers[i]
            if hasattr(layer, 'weights_gradient') and
layer.weights_gradient is not None:
                plt.figure(figsize=(10, 4))
                plt.subplot(1, 2, 1)
                plt.hist(layer.weights_gradient.flatten(), bins=30)
                plt.title(f"Weight Gradient - Layer {i}")
                plt.xlabel("Gradient value")
                plt.ylabel("Frequency")

                # plot bias gradient
```

```
                plt.subplot(1, 2, 2)
                if hasattr(layer, 'bias_gradient') and layer.bias_gradient
is not None:
                    plt.hist(layer.bias_gradient.flatten(), bins=30)
                    plt.title(f"Bias Gradient - Layer {i}")
                    plt.xlabel("Gradient value")
                    plt.ylabel("Frequency")
                else:
                    plt.text(0.5, 0.5, "No bias gradient", ha='center',
va='center')
                    plt.title(f"Bias Gradient - Layer {i}")

                plt.tight_layout()
                plt.show()
            else:
                print(f"Layer {i} tidak mempunyai weights_gradient.")
```

Nama *Attribute*	Penjelasan
layers	List yang menyimpan seluruh layer yang membentuk *neural network.*
history	Menyimpan histori nilai *training loss* dan *validation loss* selama proses *training.*
loss	Fungsi *loss* yang digunakan untuk menghitung kesalahan prediksi model.
loss_prime	Turunan dari fungsi *loss* yang digunakan saat proses *backward propagation*.

Nama *Method*	Penjelasan
__init__()	Menginisialisasi *neural network* dengan *list layer* kosong dan struktur penyimpanan *history*.
use(loss, loss_prime)	Menentukan fungsi *loss* dan turunan *loss* yang akan digunakan pada model.
add(layer)	Menambahkan *layer* baru ke dalam arsitektur *neural network*.
predict(input_data)	Melakukan *forward propagation* untuk menghasilkan prediksi dari input data.
train(x_train, y_train, x_val, y_val, epochs,	Melatih model menggunakan data *training*

batch_size, learning_rate, verbose, optimizer, beta1, beta2, epsilon)	dengan proses *forward propagation*, perhitungan *loss*, *backward propagation*, dan *update* bobot, mendukung dua jenis optimizer yaitu GD dan Adam.
get_total_regularization_loss()	Menghitung total nilai regularisasi dari seluruh layer yang memiliki *regularization*.
save(filepath)	Menyimpan model ke dalam file agar dapat digunakan kembali.
load(filepath)	Memuat model yang sebelumnya telah disimpan.
plot_weight_distribution(layer_indices)	Menampilkan distribusi bobot pada layer tertentu dalam bentuk histogram.
plot_gradient_distribution(layer_indices)	Menampilkan distribusi gradien bobot dan bias pada layer tertentu.

2.1.1.9 Kelas *FFNN*

Kelas *FFNN* merupakan kelas wrapper yang digunakan untuk membangun model FeedForward Neural Network. …

NeuralNetwork

```
class FFNN(NeuralNetwork):
    def __init__(self, layers, activations, init_func='xavier', l1=0.0,
                 l2=0.0, seed=None, seed_mode="same", mean=0.0,
                 variance=1.0, low=-0.05, high=0.05):
        super().__init__()
        if not isinstance(layers, (list, tuple)) or len(layers) < 2:
            raise ValueError("parameter 'layers' harus berupa list")
        if len(activations) != len(layers) - 1:
            raise ValueError("panjang list 'activations' harus sama dengan
jumlah layer transformasi (panjang layers - 1).")

        self._activation_functions = ActivationFunctions()
        self._initializers = Initializers()
        self.mean = mean
        self.variance = variance
        self.low = low
        self.high = high
        initializer_list = self._check_list_init_func(init_func,
len(layers) - 1)

        for i in range(len(layers) - 1):
            input_size = layers[i]
            output_size = layers[i + 1]
```

```
            init_func = self._check_init_func(initializer_list[i])
            layer_seed = None
            if seed is not None:
                if seed_mode == "same":
                    layer_seed = seed
                elif seed_mode == "incremental":
                    layer_seed = seed + i
                else:
                    raise ValueError("seed_mode harus 'same' atau
'incremental'")

            self.add(DenseLayer(input_size, output_size, init_func,
l1_lambda=l1, l2_lambda=l2, seed=layer_seed))
            activation_func, activation_prime_func =
self._check_activation_func(activations[i])
            self.add(ActivationLayer(activation_func,
activation_prime_func))

    def _check_list_init_func(self, init_func, n_layers):
        if isinstance(init_func, (list, tuple)):
            if len(init_func) != n_layers:
                raise ValueError("'init_func' harus memiliki panjang yang
sama dengan jumlah layer transformasi.")
            return list(init_func)
        return [init_func] * n_layers

    def _check_init_func(self, initializer):
        if isinstance(initializer, str):
            if initializer == 'he':
                return lambda shape, seed=None:
self._initializers.he_init(shape, seed=seed)
            if initializer == 'xavier':
                return lambda shape, seed=None:
self._initializers.xavier_init(shape, seed=seed)
            if initializer == 'normal':
                return lambda shape, seed=None:
self._initializers.normal_init(shape, mean=self.mean,
variance=self.variance, seed=seed)
            if initializer == 'uniform':
                return lambda shape, seed=None:
self._initializers.uniform_init(shape, low=self.low, high=self.high,
seed=seed)
            if initializer == 'zero':
                return lambda shape, seed=None:
self._initializers.zero_init(shape)
        raise ValueError(f"Initializer `{initializer}` tidak ditemukan.")

    def _check_activation_func(self, activation_func):
        if isinstance(activation_func, str):
            if activation_func == 'tanh':
                activation_func = 'hyperbolic_tangent'

            if activation_func in ['linear', 'relu', 'sigmoid',
'hyperbolic_tangent', 'softmax', 'leaky_relu', 'elu', 'swish']:
                prime = f"{activation_func}_prime"
                if hasattr(self._activation_functions, activation_func) and
```

```
hasattr(self._activation_functions, prime):
                    return getattr(self._activation_functions,
activation_func), getattr(self._activation_functions, prime)
        raise ValueError(f"Activation function `{activation_func}` tidak
ditemukan.")
```

Nama *Attribute*	Penjelasan
activation_functions	Objek kelas ActivationFunctions yang digunakan untuk pemilihan metode fungsi aktivasi yang digunakan
initializers	Objek kelas Initializers yang digunakan untuk pemilihan metode inisialisasi bobot pada layer model
mean	Nilai rata-rata yang digunakan pada inisialisasi distribusi normal
variance	Variansi yang digunakan pada inisialisasi distribusi normal
low	Batas bawah nilai pada inisialisasi distribusi uniform
high	Batas atas nilai pada inisialisasi distribusi uniform

Nama *Method*	Penjelasan
__init__()	Menginisialisasi kelas Neural Network beserta layer dan fungsi aktivasinya
_check_list_init_func(self, init_func, n_layers)	Melakukan validasi panjang input list metode insialisasi
_check_init_func(self, initializer)	Melakukan validasi input masing-masing metode inisialisasi
_check_activation_func(self, activation_func)	Melakukan validasi input fungsi aktivasi

2.1.2 Forward Propagation

Forward propagation merupakan proses perhitungan output dari sebuah *neural network* berdasarkan input yang diberikan. Pada tahap ini, data diproses secara bertahap dari input layer hingga mencapai output layer melalui setiap layer yang terdapat dalam jaringan.

Pada setiap layer, dilakukan transformasi terhadap data input. Pada DenseLayer, input akan dikalikan dengan bobot dan ditambahkan dengan bias menggunakan persamaan linear Z = XW + b. Hasil dari perhitungan ini kemudian diteruskan ke ActivationLayer untuk diterapkan fungsi aktivasi, sehingga model dapat menangkap hubungan non-linear dalam data.

Proses ini dilakukan secara berulang untuk setiap layer dalam jaringan hingga menghasilkan output akhir. Output tersebut merupakan hasil prediksi dari model berdasarkan input yang diberikan. Dalam implementasi yang kami buat, *forward propagation* dilakukan dengan cara mengiterasi seluruh layer secara berurutan menggunakan method forward() pada masing-masing layer.

Selain itu, *forward propagation* pada model ini telah mendukung input dalam bentuk *batch*, sehingga beberapa data dapat diproses sekaligus dalam satu iterasi. Hal ini membuat proses pelatihan menjadi lebih efisien dan sesuai dengan kebutuhan *training* modern.

2.1.3 *Backward Propagation* dan *Weight Update*

Setelah proses *forward propagation* selesai dan nilai *loss* telah dihitung, jaringan saraf masuk ke tahap *backward propagation.* Tahap ini adalah proses pembelajaran di mana model belajar dari kesalahan untuk memperbaiki prediksi pada iterasi selanjutnya.

2.1.3.1 *Backward Propagation*

Backward propagation merupakan algoritma untuk menghitung gradien dari *loss function* terhadap setiap bobot dan bias dalam jaringan. Proses ini dilakukan secara mundur atau *backward* dari lapisan output hingga kembali ke lapisan input. Dalam implementasi ini, kami menggunakan prinsip aturan rantai atau *chain rule.* Jaringan menghitung seberapa besar perubahan pada suatu bobot akan memengaruhi nilai *loss.* Pada implementasi yang kami buat, proses *backward* terjadi pada *ActivationLayer* dan juga *DenseLayer.*

Fungsi *backward* pada *DenseLayer* menggunakan metode *batch* dimana gradien dibagi dengan ukuran *batch* untuk mendapatkan estimasi gradien yang lebih stabil. Tanpa pembagian dengan jumlah *batch,* nilai gradien akan menjadi besar seiring dengan bertambahnya ukuran *batch* yang dapat menyebabkan proses *weight update* menjadi tidak stabil. Sebelum bobot diperbarui, gradien juga dikalikan dengan faktor pengali dari L1 dan L2 untuk memastikan setiap langkah pembaruan bobot selalu memenuhi regularisasi agar nilai parameternya tidak terlalu besar.

2.1.3.2 *Weight Update*

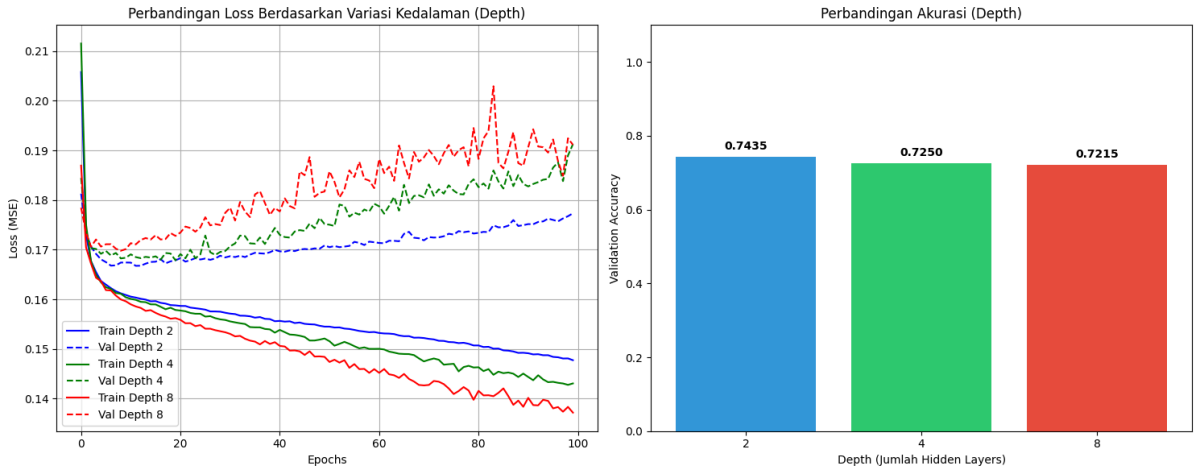
Setelah nilai gradien untuk setiap parameter ditemukan melalui *backpropagation,* selanjutnya akan dilakukan pembaruan nilai bobot dan bias. Algoritma yang digunakan adalah *stochastic gradient descent.* Pembaruan dilakukan dengan menggeser nilai bobot lama ke arah yang berlawanan dengan arah gradien menuju titik minimum fungsi *loss.* Rumus pembaruan bobot adalah $w\,new = w\,old - \eta(dLoss/dw)$.

Pada pembaruan bobot, apabila nilai *learning rate* terlalu tinggi, langkah pembaruan akan terlalu drastis sehingga menyebabkan model bisa saja melewatkan titik minimum dan membuat grafik *loss* menjadi tidak stabil. Sebaliknya, apabila nilai *learning rate* terlalu rendah, pembaruan bobot akan berlangsung sangat lambat sehingga model membutuhkan waktu *epoch* yang jauh lebih banyak untuk konvergen atau berisiko terjebak pada *local minima.*

2.2 Hasil Pengujian

2.2.1 Pengujian *Hyperparameter*

2.2.1.1 Pengaruh Depth dan Width



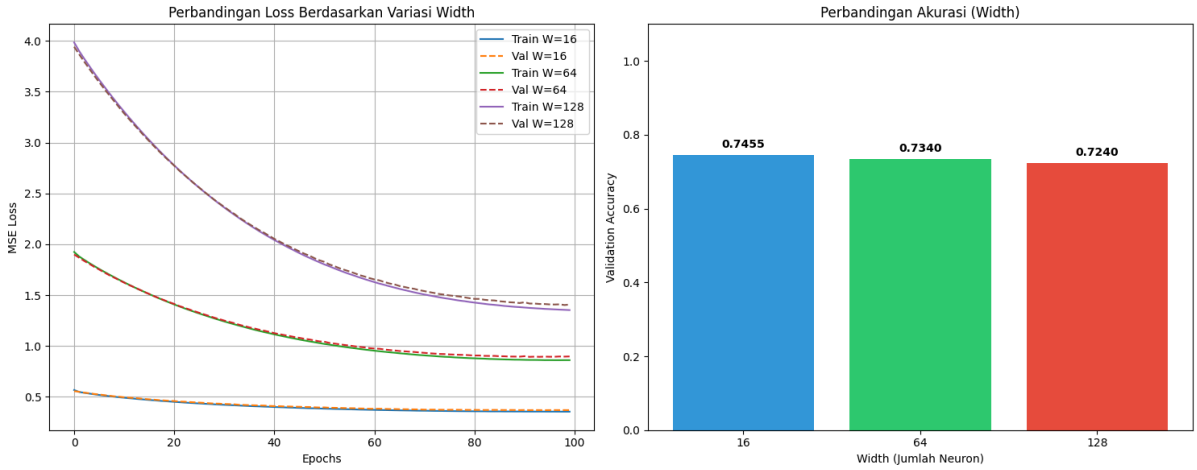
Gambar 2.2.1.1.1 Grafik Pengujian *Depth*

Berdasarkan hasil pengujian pada variasi kedalaman jaringan, grafik *loss* menunjukkan fenomena overfitting yang cukup signifikan pada ketiga model. Hal ini ditandai dengan adanya divergensi antara kurva *train loss* dan *validation loss.* Kondisi ini mengindikasikan bahwa model cenderung menghafal detail serta *noise* pada data baru. Semakin dalam konfigurasi jaringan, semakin terlihat ketidakstabilan yang paling tinggi dibanding yang lain.

Ditinjau dari sisi akurasi, hasil eksperimen menunjukkan bahwa penambahan *hidden layer* tidak selalu meningkatkan performa klasifikasi pada dataset ini. Model dengan *depth 2* justru menghasilkan akurasi validasi paling tinggi. Penurunan akurasi seiring dengan bertambahnya

kedalaman jaringan ini menandakan bahwa arsitektur yang terlalu kompleks untuk jumlah fitur yang tersedia justru membuat *overfitting*.

Meskipun secara teoritis penambahan kedalaman jaringan memungkinkan model untuk belajar lebih baik namun hasil eksperimen ini menunjukkan adanya batas optimal. Penambahan layer yang tidak diimbangi dengan jumlah data yang besar atau pemilihan fitur yang baik malah akan membuat *overfitting*. Ada kemungkinan apabila menggunakan variasi *hyperparameter* yang lain bisa menghasilkan model yang lebih baik.



Gambar 2.2.1.1.2 Grafik Pengujian Width

Berdasarkan grafik perbandingan *loss* berdasarkan variasi *width* atau lebar, terlihat bahwa skala *loss* awal pada model dengan *width* yang besar pada epoch pertama lebih tinggi dibandingkan dengan *width* yang sedang dan kecil. Hal ini terjadi karena semakin banyak jumlah neuron, maka semakin banyak bobot acak yang diinisialisasi sehingga kesalahan awal kumulatifnya lebih besar.

Ditinjau dari kelengkungan kurva, model dengan *width* terbesar (berwarna ungu) memiliki kelengkungan yang paling curam. Hal ini menunjukkan bahwa jaringan yang lebar memiliki kecepatan belajar yang tinggi karena memiliki kapasitas memori yang besar. Sedangkan dapat dilihat pada model dengan *width* kecil (berwarna biru) grafiknya lebih landai dan cepat konstan karena model sederhana dan kapasitasnya terbatas. Meskipun angka mulainya berbeda, namun semua variasi menunjukkan tren menurun yang stabil yang artinya tidak ada *overfitting* yang signifikan.

Ditinjau dari akurasi, model dengan *width* lebih kecil memiliki akurasi yang lebih besar dibandingkan yang lain. Hal ini menandakan bahwa jaringan yang lebih sederhana justru lebih efektif menghindari *overfitting* pada dataset ini karena model yang terlalu lebar

cenderung memiliki kompleksitas yang berlebihan sehingga performa validasinya bisa menurun.

2.2.1.2 Pengaruh Fungsi Aktivasi Hidden Layer

Pengujian fungsi aktivasi dilakukan untuk menganalisis pengaruh pemilihan fungsi aktivasi terhadap performa model *neural network.* Pengujian ini menggunakan model dengan 3 layer transformasi dan 7 variasi fungsi aktivasi pada salah satu hidden layer. Variasi fungsi aktivasi yang digunakan adalah linear, relu, sigmoid, tanh, leaky relu, elu, dan swish. Selanjutnya untuk setiap variasi, kami menampilkan accuracy dan f1 score, visualisasi training loss dan validation loss, weight distribution, serta gradient distribution.

```
Testing activation function: linear
  Accuracy: 0.7515, Macro F1: 0.7328

Testing activation function: relu
  Accuracy: 0.7530, Macro F1: 0.7349

Testing activation function: sigmoid
  Accuracy: 0.7565, Macro F1: 0.7380

Testing activation function: tanh
  Accuracy: 0.7520, Macro F1: 0.7334
```

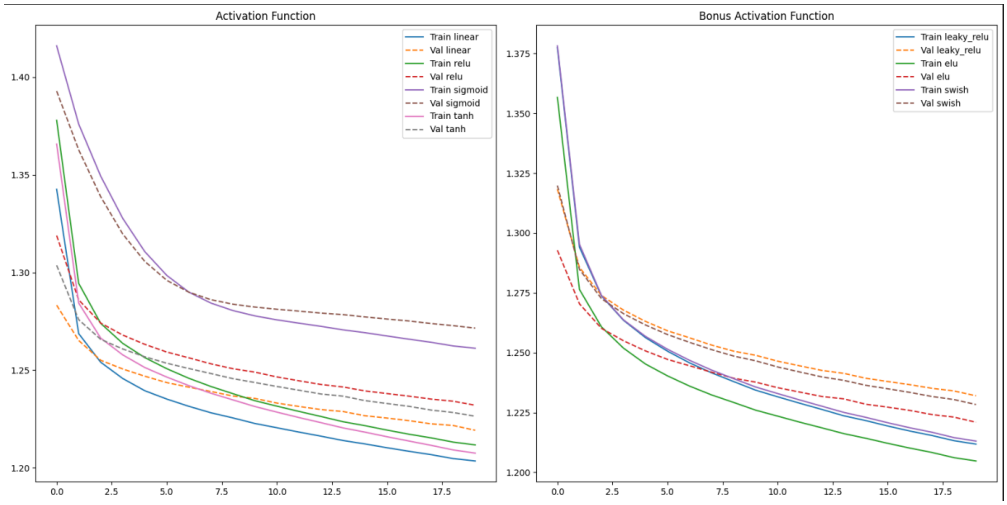
```
Testing activation function: leaky_relu
  Accuracy: 0.7535, Macro F1: 0.7356

Testing activation function: elu
  Accuracy: 0.7585, Macro F1: 0.7399

Testing activation function: swish
  Accuracy: 0.7585, Macro F1: 0.7396
```

Gambar 2.2.1.2.1 Perbandingan Evaluasi Model Berdasarkan Variasi Fungsi Aktivasi

Berdasarkan akurasi dan skor f1, fungsi aktivasi memiliki pengaruh terhadap performa klasifikasi model. Model yang menghasilkan akurasi tertinggi (0,7585) adalah model dengan fungsi aktivasi elu (f1 0,7399) dan swish (f1 0,7396). Sebaliknya, model dengan performa terendah adalah model dengan fungsi aktivasi linear (akurasi 0,7515 dan f1 score 0,7328).

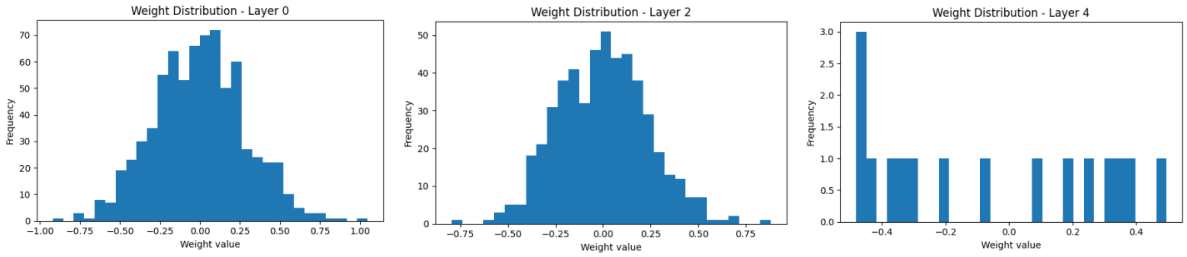


Gambar 2.2.1.4.2 Grafik *Training dan Validation Loss* Berdasarkan Variasi Fungsi Aktivasi

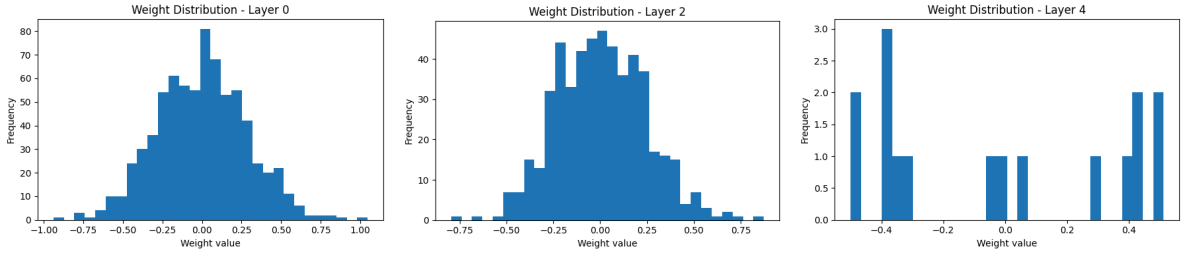
Berdasarkan grafik *training loss*, terlihat bahwa seluruh variasi fungsi aktivasi menunjukkan tren penurunan loss seiring bertambahnya jumlah epoch. Hal ini menunjukkan bahwa model berhasil mempelajari pola data dengan baik pada seluruh konfigurasi aktivasi. Fungsi aktivasi

sigmoid menunjukkan penurunan yang cukup landai dibandingkan fungsi aktivasi lainnya dan menghasilkan nilai loss yang relatif paling tinggi dibandingkan variasi lainnya. Hal ini mengindikasikan proses pembelajaran yang kurang optimal pada fungsi aktivasi sigmoid.

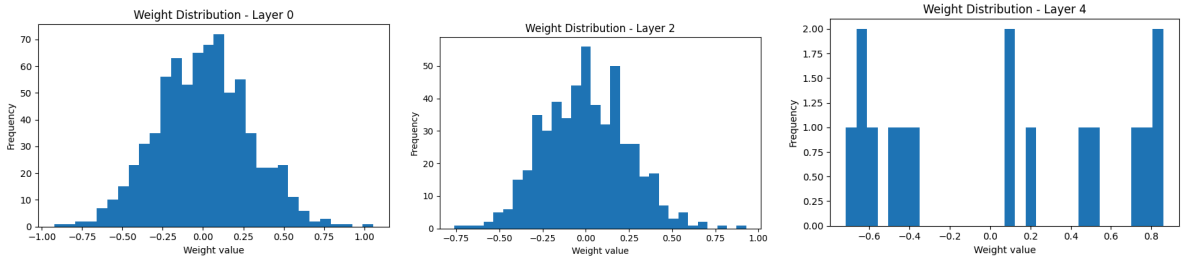
Pada grafik validation loss, pola yang serupa juga dapat ditemukan. Sebagian besar fungsi aktivasi menunjukkan penurunan yang cukup signifikan di awal epoch, kemudian diikuti dengan penurunan yang lebih stabil. Model dengan fungsi aktivasi elu dan linear cenderung menghasilkan nilai validation loss yang lebih rendah dibandingkan variasi lainnya. Sementara itu, fungsi sigmoid menghasilkan validation loss yang relatif paling tinggi dan cenderung lebih lambat dalam penurunannya. Selain itu, tidak terlihat adanya perbedaan yang signifikan antara kurva training dan validation loss pada seluruh variasi, yang menunjukkan bahwa tidak terjadi overfitting yang berarti selama proses pelatihan.



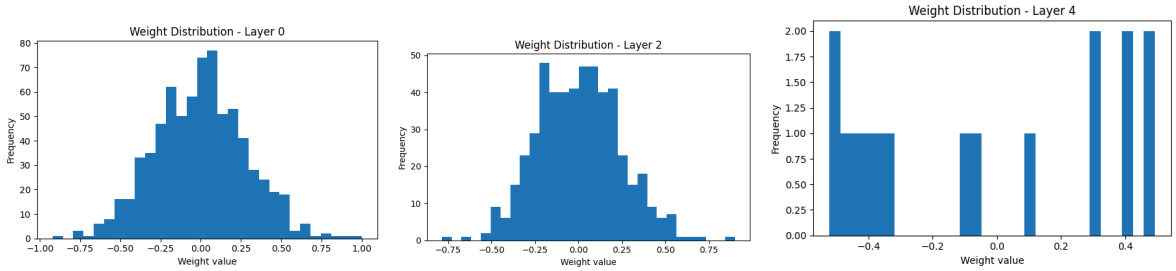
Gambar 2.2.1.3.3 Grafik *Weight Distribution* pada Model dengan Fungsi Aktivasi Linear



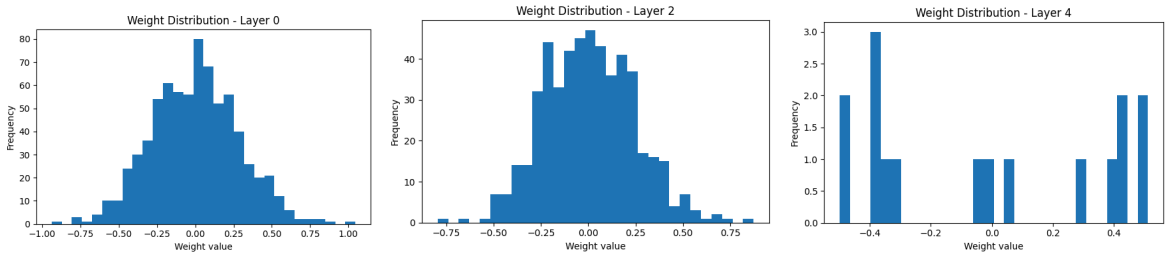
Gambar 2.2.1.3.4 Grafik *Weight Distribution* pada Model dengan Fungsi Aktivasi ReLU



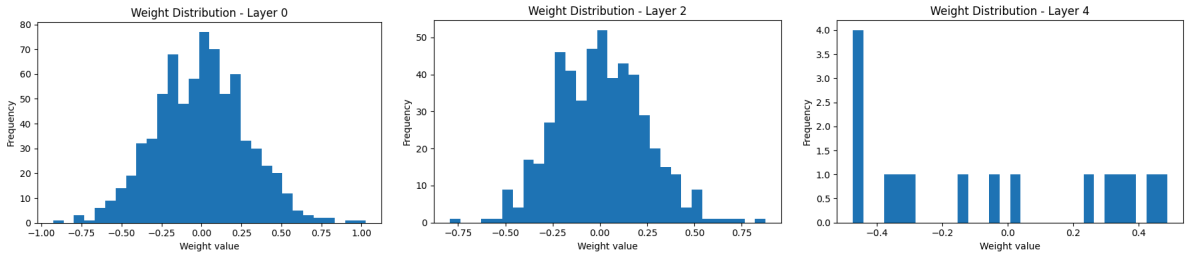
Gambar 2.2.1.3.5 Grafik *Weight Distribution* pada Model dengan Fungsi Aktivasi Sigmoid



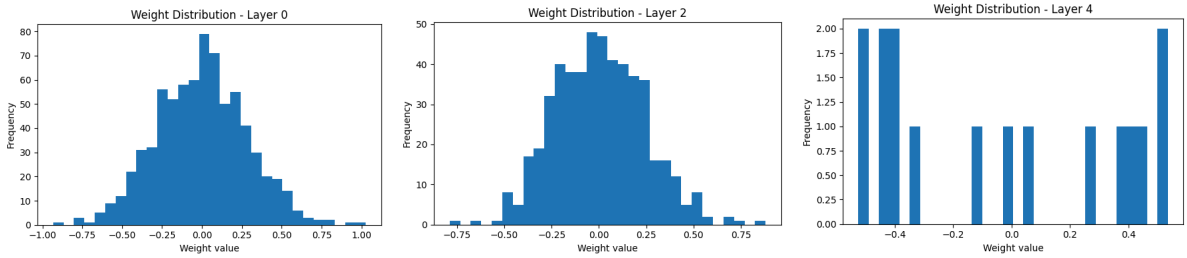
Gambar 2.2.1.3.6 Grafik *Weight Distribution* pada Model dengan Fungsi Aktivasi Tanh



Gambar 2.2.1.3.7 Grafik *Weight Distribution* pada Model dengan Fungsi Aktivasi Leaky ReLU



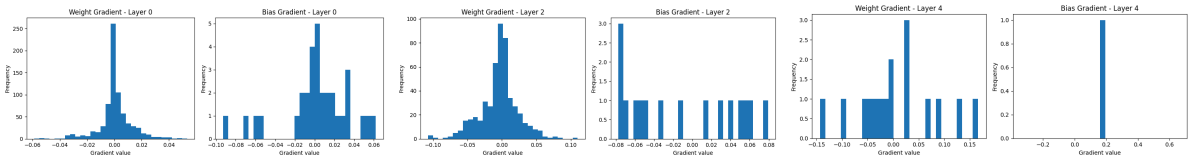
Gambar 2.2.1.3.8 Grafik Weight Distribution pada Model dengan Fungsi Aktivasi ELU



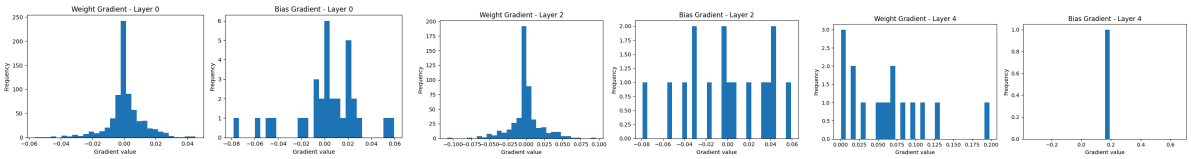
Gambar 2.2.1.3.9 Grafik *Weight Distribution* pada Model dengan Fungsi Aktivasi Swish

Grafik tersebut berturut turut adalah linear, relu, sigmoid, tanh, leaky relu, elu, dan swish. Berdasarkan distribusi bobot, terlihat bahwa seluruh fungsi aktivasi menghasilkan distribusi yang terpusat di sekitar nol pada layer awal, menunjukkan proses pembelajaran yang stabil. Seluruh variasi fungsi aktivasi menunjukkan distribusi yang relatif mirip sehingga dapat

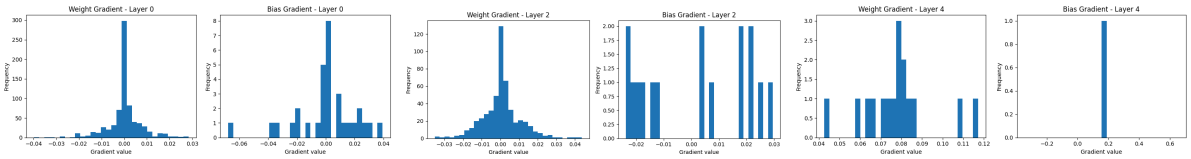
disimpulkan bahwa variasi fungsi aktivasi tidak memberikan pengaruh besar terhadap pola distribusi bobot.



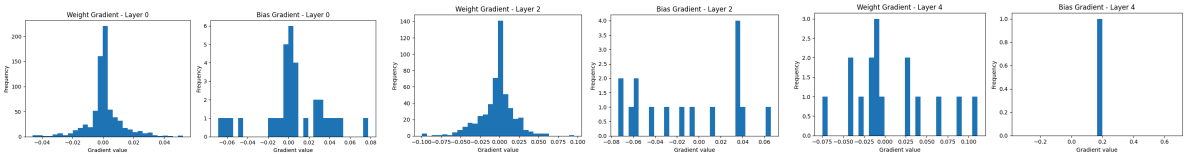
Gambar 2.2.1.3.10 Grafik *Gradient Distribution* pada Model dengan Fungsi Aktivasi Linear



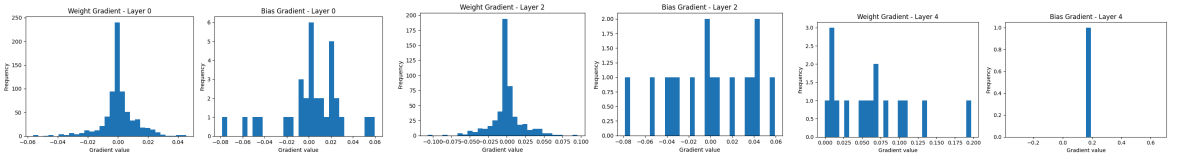
Gambar 2.2.1.3.11 Grafik *Gradient Distribution* pada Model dengan Fungsi Aktivasi ReLU



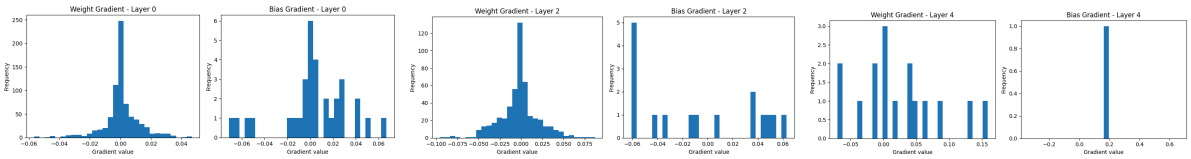
Gambar 2.2.1.3.12 Grafik *Gradient Distribution* pada Model dengan Fungsi Aktivasi Sigmoid



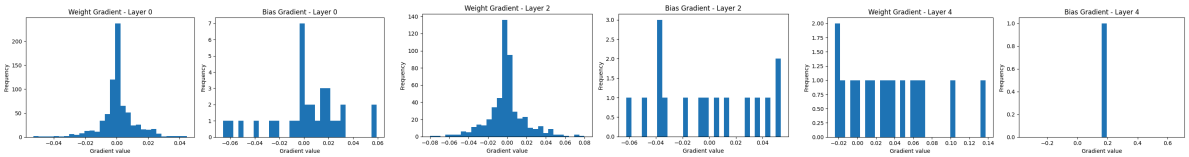
Gambar 2.2.1.3.13 Grafik *Gradient Distribution* pada Model dengan Fungsi Aktivasi Tanh



Gambar 2.2.1.3.14 Grafik *Gradient Distribution* pada Model dengan Fungsi Aktivasi Leaky ReLU



Gambar 2.2.1.3.15 Grafik *Gradient Distribution* pada Model dengan Fungsi Aktivasi ELU



Gambar 2.2.1.3.16 Grafik *Gradient Distribution* pada Model dengan Fungsi Aktivasi Swish

Grafik tersebut berturut turut menunjukkan distribusi gradien pada model dengan fungsi aktivasi linear, relu, sigmoid, tanh, leaky relu, elu, dan swish. Berdasarkan distribusi bobot dan gradien, terlihat bahwa seluruh variasi fungsi aktivasi menghasilkan pola distribusi yang relatif mirip, yaitu terpusat di sekitar nilai nol. Hal ini menunjukkan bahwa proses pembelajaran berjalan stabil tanpa adanya penyimpangan nilai yang ekstrem baik pada bobot maupun gradien. Perbedaan distribusi antar fungsi aktivasi tidak terlalu signifikan sehingga dapat disimpulkan bahwa variasi fungsi aktivasi tidak memberikan pengaruh besar terhadap pola distribusi gradien model.

2.2.1.3 Pengaruh *Learning Rate*

Pengujian ini dilakukan untuk menganalisis pengaruh learning rate terhadap performa model neural network. Pengujian ini menggunakan model dengan 3 layer transformasi dengan fungsi aktivasi relu pada hidden layer dan sigmoid pada output layer, serta variasi learning rate sebesar 0.001, 0.01, dan 0.1. Untuk setiap variasi, dilakukan evaluasi menggunakan accuracy dan macro f1 score, serta visualisasi training loss dan validation loss, dan analisis distribusi bobot dan gradien.

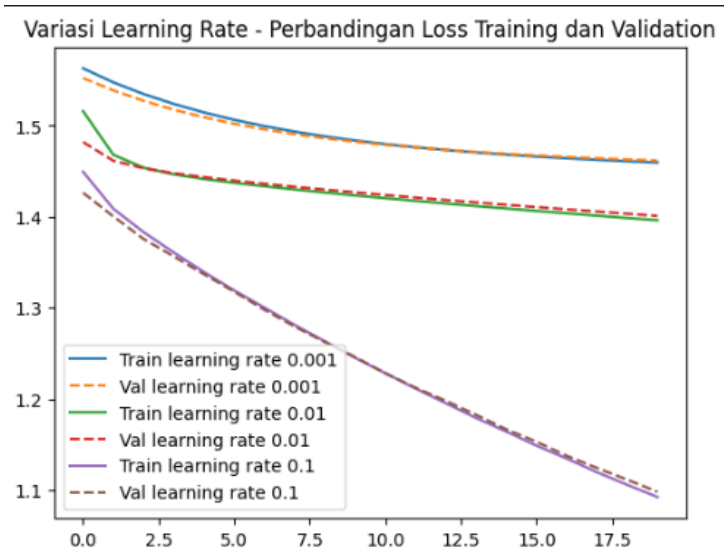
```
Testing learning rate: learning rate 0.001
  Accuracy: 0.7185, Macro F1: 0.6885

Testing learning rate: learning rate 0.01
  Accuracy: 0.7470, Macro F1: 0.7246

Testing learning rate: learning rate 0.1
  Accuracy: 0.7480, Macro F1: 0.7315
```

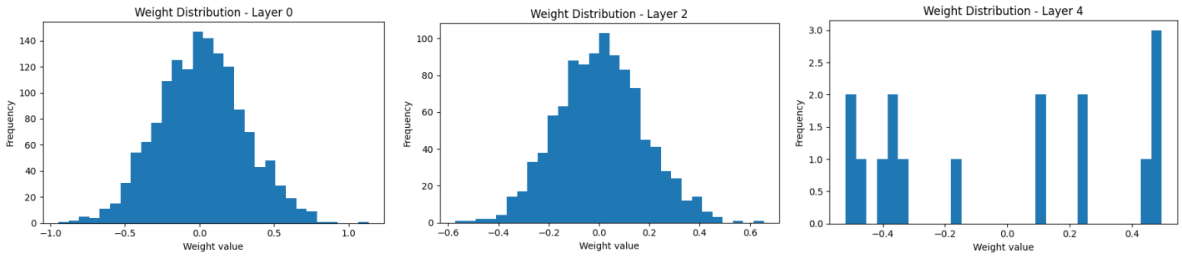
Gambar 2.2.1.3.1 Perbandingan Evaluasi Model Berdasarkan Variasi Learning Rate

Berdasarkan akurasi dan skor f1, terlihat bahwa learning rate memberikan pengaruh yang signifikan terhadap performa model. Learning rate 0.001 menghasilkan performa terendah dengan accuracy 0.7185 dan macro f1 0.6885, yang menunjukkan bahwa proses pembelajaran berjalan terlalu lambat. Sementara itu, learning rate 0.01 dan 0.1 menghasilkan performa yang lebih baik, dengan 0.1 memberikan accuracy tertinggi yaitu 0.7480 dan macro f1 sebesar 0.7315, diikuti oleh 0.01 dengan accuracy 0.7470 dan macro f1 0.7246. Hal ini menunjukkan bahwa learning rate yang lebih besar mampu membantu model mencapai performa yang lebih optimal dalam jumlah epoch yang sama.

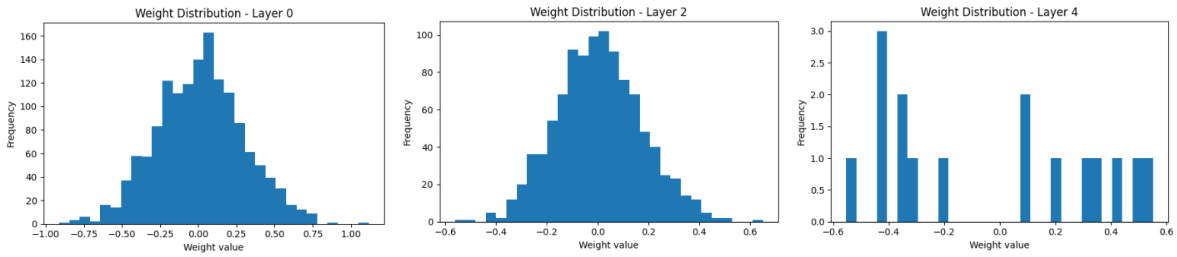


Gambar 2.2.1.3.2 Grafik *Training dan Validation Loss*

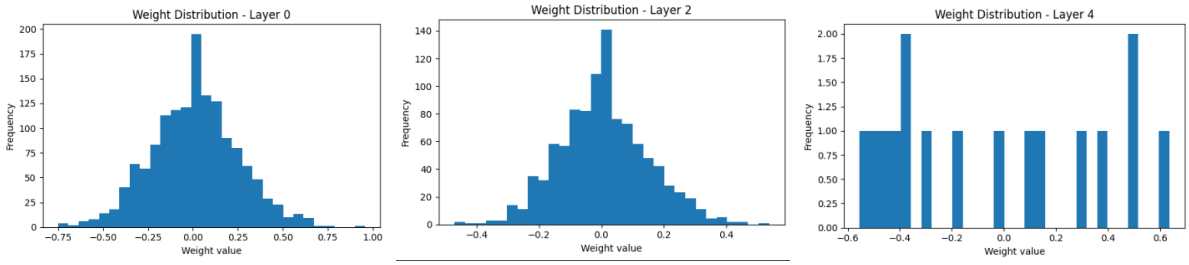
Berdasarkan grafik training loss dan validation loss, terlihat bahwa seluruh variasi learning rate menunjukkan tren penurunan loss seiring bertambahnya epoch. Namun, terdapat perbedaan yang cukup jelas pada kecepatan penurunannya. Learning rate 0.1 menunjukkan penurunan loss yang paling cepat dan mencapai nilai loss terendah dibandingkan variasi lainnya. Learning rate 0.01 juga menunjukkan penurunan yang cukup cepat namun berada sedikit di atas 0.1. Sementara itu, learning rate 0.001 memiliki penurunan yang paling lambat dan cenderung berada pada nilai loss yang lebih tinggi sepanjang proses training. Selain itu, kurva training dan validation loss pada ketiga variasi cenderung berdekatan, yang menunjukkan bahwa tidak terjadi overfitting yang signifikan. Secara keseluruhan, learning rate yang lebih besar dalam pengujian ini mampu mempercepat penurunan loss dan menghasilkan performa model yang lebih optimal.



Gambar 2.2.1.3.3 Grafik *Weight Distribution* pada Model dengan Learning Rate 0,001

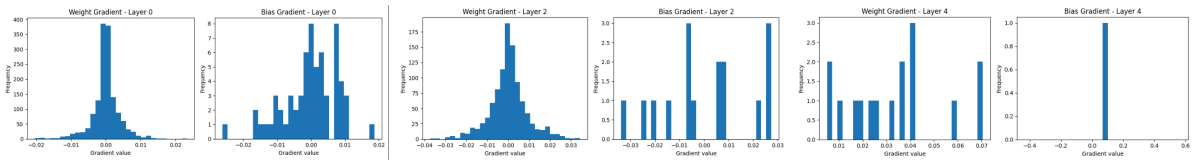


Gambar 2.2.1.3.4 Grafik *Weight Distribution* pada Model dengan Learning Rate 0,01

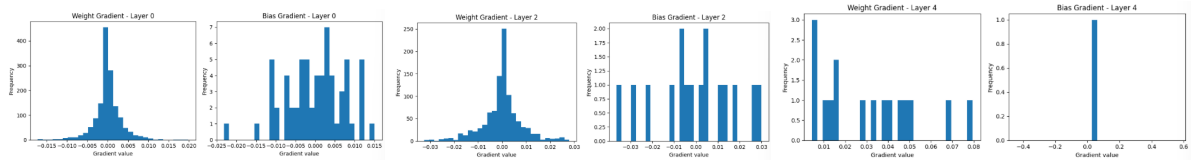


Gambar 2.2.1.3.5 Grafik *Weight Distribution* pada Model dengan Learning Rate 0,1

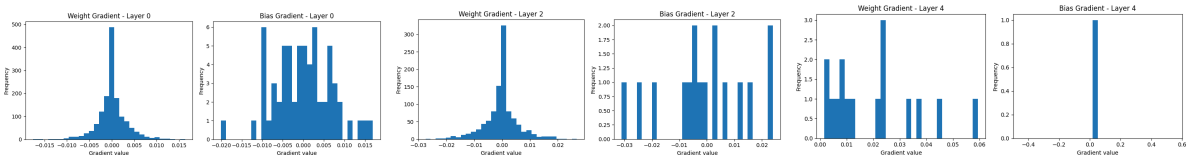
Berdasarkan distribusi bobot, terlihat bahwa pada seluruh variasi learning rate, bobot pada layer awal cenderung terpusat di sekitar nol, yang menunjukkan proses pembelajaran berjalan stabil. Pada learning rate 0.001, distribusi bobot terlihat lebih menyebar. Sementara itu, pada learning rate 0.01, distribusi menjadi lebih terpusat dan simetris, menunjukkan kondisi pembelajaran yang lebih stabil dan optimal. Pada learning rate 0.1, distribusi bobot terlihat paling terkonsentrasi dengan puncak yang lebih tajam. Hal ini mengindikasikan proses pembaruan bobot yang lebih agresif dan konvergensi yang lebih cepat. Secara keseluruhan, learning rate yang lebih besar menghasilkan distribusi bobot yang lebih terkonsentrasi dan cepat konvergen, sedangkan learning rate kecil menghasilkan distribusi yang lebih menyebar dan lambat beradaptasi.



Gambar 2.2.1.3.6 Grafik *Gradient Distribution* pada Model dengan Learning Rate 0,001



Gambar 2.2.1.3.7 Grafik *Gradient Distribution* pada Model dengan Learning Rate 0,01



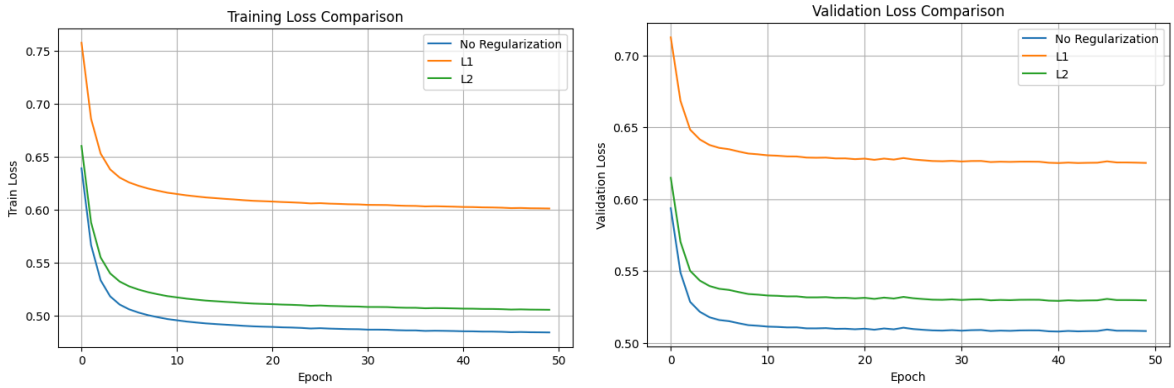
Gambar 2.2.1.3.7 Grafik *Gradient Distribution* pada Model dengan Learning Rate 0,1

Berdasarkan distribusi gradien, terlihat bahwa pola distribusi pada ketiga variasi relatif mirip, yaitu terpusat di sekitar nol. Pada learning rate 0.1, frekuensi nilai gradien di sekitar nol terlihat lebih tinggi dibandingkan dengan learning rate 0.001, ditunjukkan oleh puncak distribusi yang lebih tajam. Hal ini mengindikasikan bahwa pada learning rate yang lebih besar terdapat lebih banyak gradien yang bernilai kecil.

2.2.1.4 Pengaruh Regularisasi

Pada pengujian ini dilakukan analisis terhadap pengaruh penggunaan regularisasi terhadap performa dan karakteristik bobot pada model *neural network*. Pada pengujian ini dilakukan tiga skenario pelatihan model, yaitu model tanpa regularisasi, model dengan regularisasi L1, dan model dengan regularisasi L2. Regularisasi L1 bekerja dengan menambahkan penalti berupa nilai absolut dari bobot sehingga mendorong beberapa bobot menjadi sangat kecil atau bahkan nol. Sementara itu, regularisasi L2 menambahkan penalti berupa kuadrat bobot sehingga mengecilkan nilai bobot tetapi tidak nol. Dengan adanya penalti tersebut, model diharapkan dapat menghasilkan bobot yang lebih stabil dan mengurangi risiko *overfitting*.

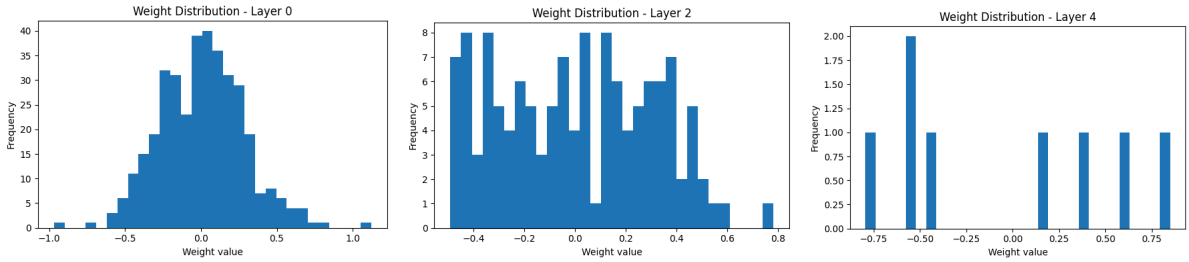
Untuk menganalisis pengaruh regularisasi, dilakukan visualisasi *training loss* dan *validation loss* selama proses pelatihan serta distribusi bobot dan distribusi gradien pada beberapa *layer* model setelah proses training selesai. Grafik *training loss* dan *validation loss* digunakan untuk melihat bagaimana performa model selama proses *training* dan apakah model mengalami *overfitting* atau tidak. Sementara itu, grafik distribusi bobot menunjukkan bagaimana nilai bobot tersebar pada setiap *layer*, sedangkan grafik distribusi gradien menunjukkan bagaimana perubahan bobot yang terjadi selama proses *training*.



Gambar 2.2.1.4.1 Grafik *Training Loss* dan *Validation Loss* Uji Regularisasi

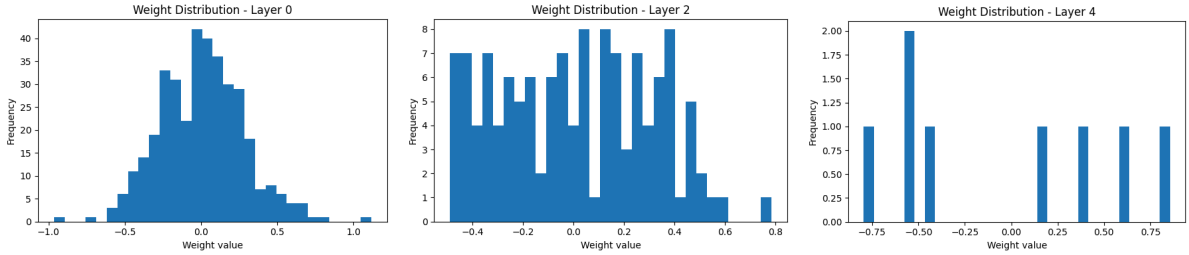
Berdasarkan grafik *training loss*, ketiga model sama-sama menunjukkan tren penurunan *loss* seiring bertambahnya jumlah *epoch*, yang menandakan bahwa proses pelatihan berjalan dengan baik. Namun, terlihat bahwa model dengan regularisasi L1 memiliki nilai *training loss* yang paling tinggi, sedangkan model tanpa regularisasi menghasilkan *training loss* paling rendah. Untuk model dengan regularisasi L2 berada sangat dekat dengan model tanpa regularisasi, tetapi sedikit lebih tinggi. Hal ini menunjukkan bahwa pada konfigurasi pengujian ini, penalti dari L1 membatasi pembelajaran model lebih kuat dibandingkan L2.

Pada grafik *validation loss*, polanya sama seperti pada grafik *training loss*. Model tanpa regularisasi menghasilkan *validation loss* terendah, model L2 berada sedikit diatasnya, dan model L1 memiliki *validation loss* paling tinggi. Kedekatan kurva antara model tanpa regularisasi dan L2 menunjukkan bahwa penambahan regularisasi L2 hanya memberikan pengaruh kecil terhadap performa generalisasi, sementara regularisasi L1 memberi pembatasan yang lebih kuat sehingga nilai *loss* menjadi lebih besar.



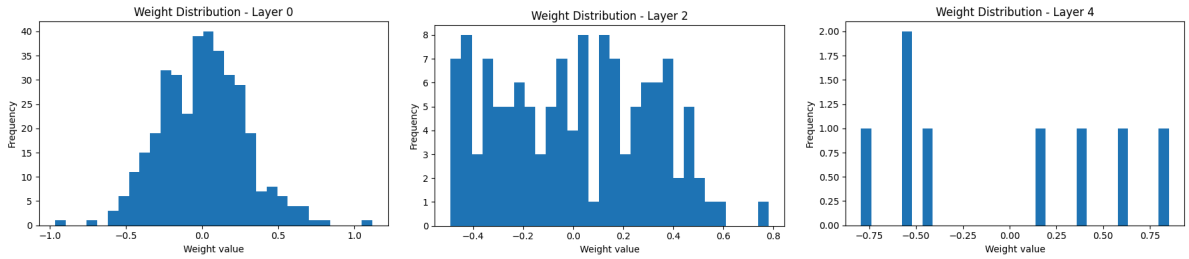
Gambar 2.2.1.4.2 Grafik *Weight Distribution* pada Model Tanpa Regularisasi

Pada model tanpa regularisasi, distribusi bobot pada setiap layer terlihat menyebar di sekitar nol dengan rentang nilai yang relatif lebar. Pola ini menunjukkan bahwa bobot dapat berkembang lebih bebas selama proses pelatihan karena tidak ada penalti tambahan yang menahan besarannya. Pada *layer* awal, distribusi tampak lebih padat di sekitar nol, sedangkan pada *layer* yang lebih dalam, jumlah bobot lebih sedikit sehingga histogram tampak lebih jarang. Model ini mempertahankan variasi bobot yang lebih bebas dibandingkan model dengan regularisasi.



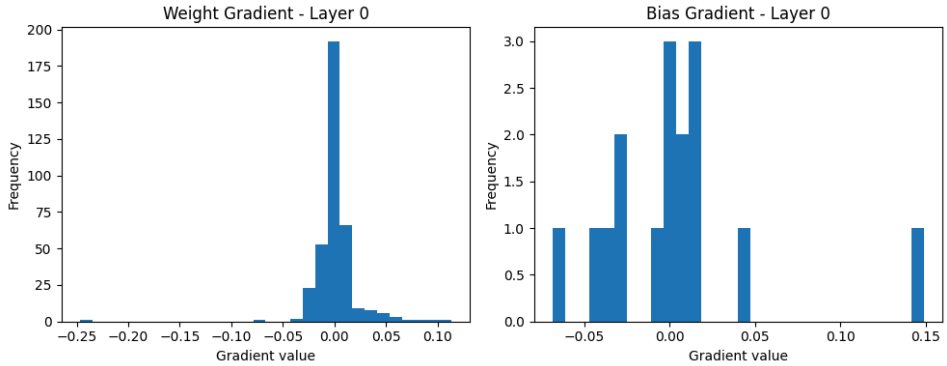
Gambar 2.2.1.4.3 Grafik *Weight Distribution* pada Model Regularisasi L1

Pada model dengan regularisasi L1, distribusi bobot masih terpusat di sekitar nol, tetapi terlihat adanya perubahan pola dibandingkan model lainnya, terutama pada beberapa bobot yang terdorong menjadi lebih kecil. Meskipun tidak semua bobot menjadi nol, regularisasi L1 tetap menunjukkan efek *sparsity* yang lebih terasa dibandingkan L2. Hal ini sejalan dengan sifat L1 yang memberikan penalti berdasarkan nilai absolut bobot, sehingga cenderung menekan bobot lebih kuat dan membuat distribusinya sedikit berbeda.

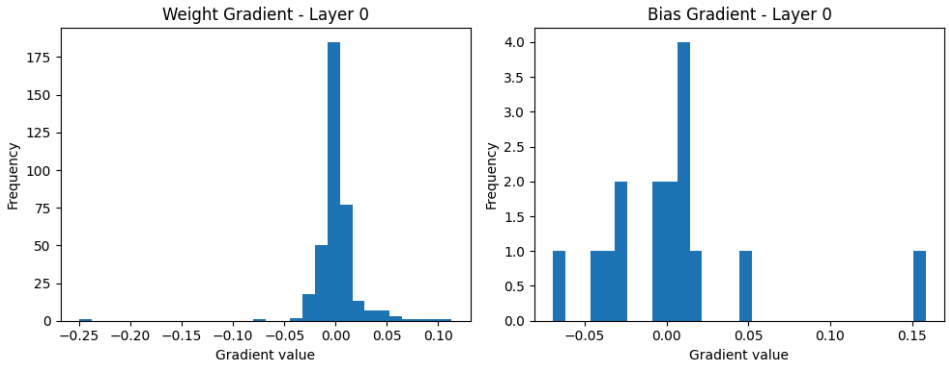


Gambar 2.2.1.4.4 Grafik *Weight Distribution* pada Model Regularisasi L2

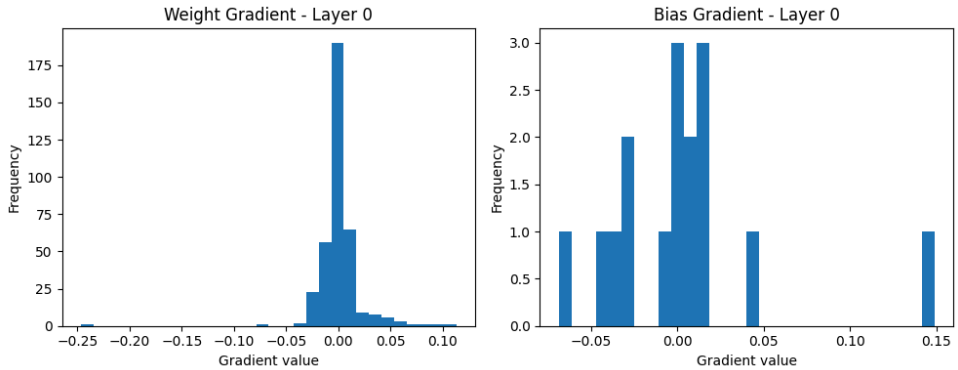
Pada model dengan regularisasi L2, distribusi bobot juga menunjukkan pola yang sangat mirip dengan model tanpa regularisasi. Bobot tetap terkonsentrasi di sekitar nol, tetapi tidak terlihat perubahan bentuk distribusi yang mencolok. Hal ini menunjukkan bahwa pada konfigurasi pengujian ini, regularisasi L2 lebih berperan sebagai mekanisme untuk menahan bobot agar tidak tumbuh terlalu besar, tanpa mengubah pola penyebaran bobot secara drastis. Dengan demikian, Pengaruh L2 terhadap distribusi bobot pada pengujian ini relatif kecil dibandingkan L1.



Gambar 2.2.1.4.5 Weight Gradient Layer 0 pada Model Tanpa Regularisasi



Gambar 2.2.1.4.6 Weight Gradient Layer 0 pada Model Regularisasi L1

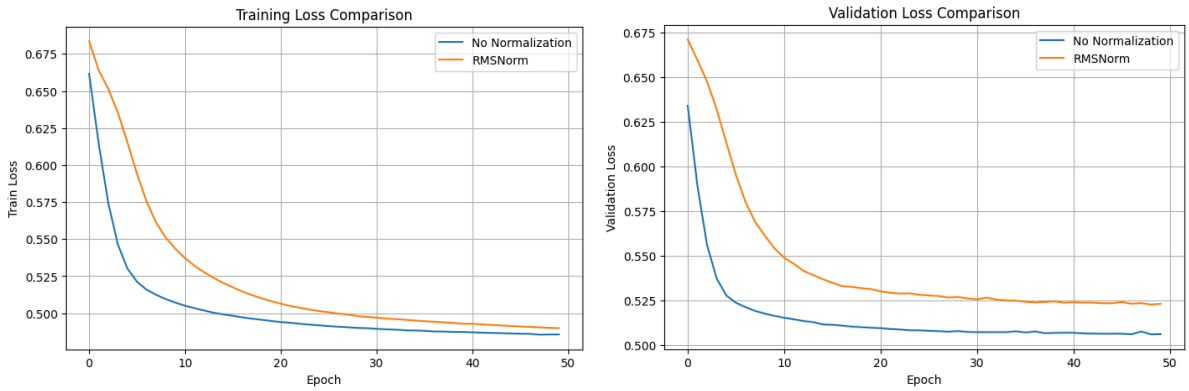


Gambar 2.2.1.4.7 Weight Gradient Layer 0 pada Model Regularisasi L2

Berdasarkan grafik distribusi gradien, model tanpa regularisasi dan L2 memperlihatkan pola yang hampir sama, baik pada distribusi gradien bobot maupun gradien bias. Sementara itu, model dengan regularisasi L1 menunjukkan sedikit perbedaan pola, meskipun secara umum gradien ketiga model tetap terpusat di sekitar nol. Hal ini menunjukkan bahwa pada pengujian ini, regularisasi L2 tidak banyak mengubah dinamika pembaruan parameter dibandingkan model tanpa regularisasi, sedangkan L1 memberi pengaruh yang sedikit lebih nyata terhadap proses *update* bobot.

2.2.1.5 Pengaruh Normalisasi RMSNorm

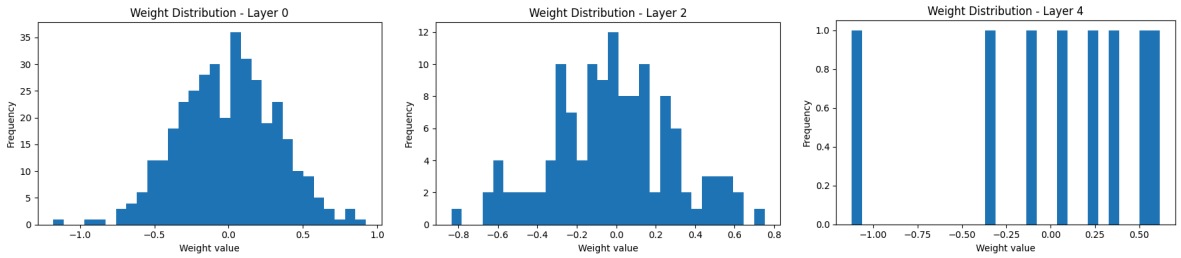
Pada pengujian ini dilakukan analisis terhadap pengaruh penggunaan normalisasi RMSNorm terhadap performa model *neural network*. RMSNorm adalah metode normalisasi yang bekerja dengan menstabilkan skala aktivasi pada setiap layer tanpa mengurangi *mean* seperti pada *batch normalization*. Untuk menganalisis pengaruh RMSNorm, dilakukan dua skenario pelatihan model, yaitu model tanpa normalisasi dan model dengan RMSNorm. Evaluasi dilakukan dengan membandingkan grafik *training loss* dan *validation loss* selama proses pelatihan, serta distribusi bobot dan gradien pada beberapa layer.



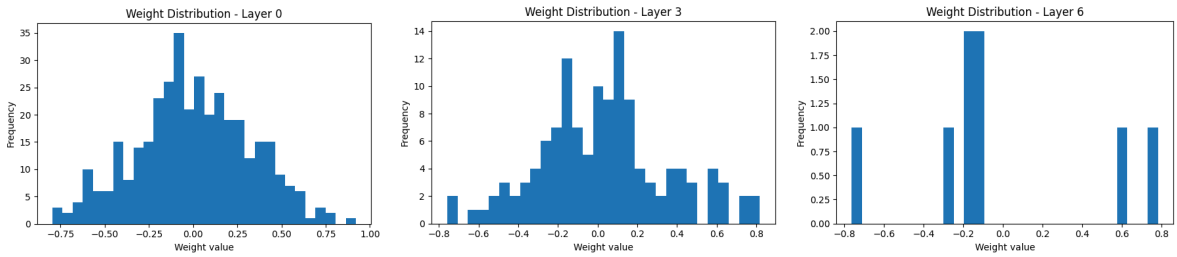
Gambar 2.2.1.5.1 Grafik *Training Loss* dan *Validation Loss* Uji RMSNorm

Berdasarkan grafik *training loss*, terlihat bahwa kedua model sama-sama menunjukkan tren penurunan *loss* seiring bertambahnya jumlah *epoch*. Namun, model tanpa normalisasi memiliki penurunan *loss* yang lebih cepat dan mencapai nilai *loss* yang lebih rendah dibandingkan model dengan RMSNorm. Kurva RMSNorm terlihat lebih landai dan berada sedikit di atas model tanpa normalisasi sepanjang proses *training*.

Pada grafik *validation loss*, pola yang sama juga terlihat. Model tanpa normalisasi menghasilkan nilai *validation loss* yang lebih rendah dibandingkan model dengan RMSNorm. Selain itu, kurva RMSNorm cenderung lebih stabil namun tetap berada di atas model tanpa normalisasi, yang menunjukkan bahwa pada konfigurasi pengujian ini, RMSNorm belum memberikan peningkatan performa generalisasi yang signifikan.

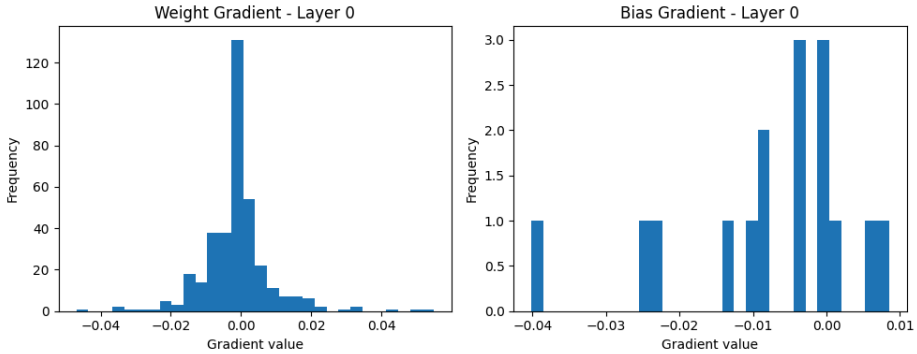


Gambar 2.2.1.5.2 Grafik *Weight Distribution* Model Tanpa Normalisasi

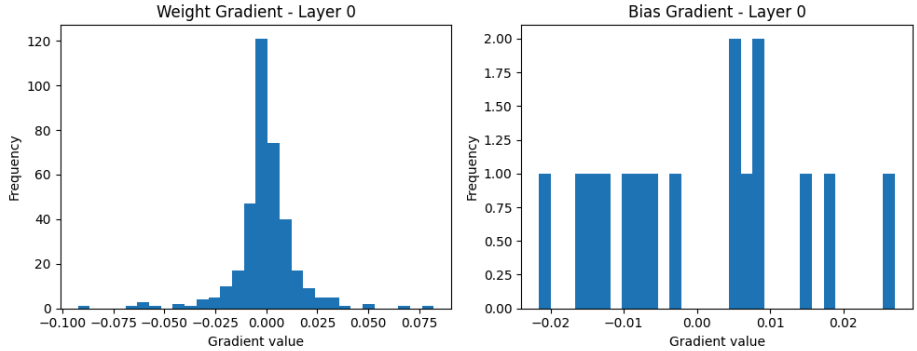


Gambar 2.2.1.5.3 Grafik *Weight Distribution* Model dengan RMSNorm

Berdasarkan distribusi bobot, kedua model menunjukkan pola penyebaran bobot yang relatif mirip, yaitu terpusat di sekitar nol. Namun, pada model dengan RMSNorm, distribusi bobot terlihat sedikit lebih terkontrol dengan rentang yang tidak terlalu jauh dibandingkan model tanpa normalisasi. Hal ini menunjukkan bahwa RMSNorm membantu menjaga kestabilan nilai bobot selama proses *training*, meskipun perbedaannya tidak terlalu signifikan.



Gambar 2.2.1.5.4 Grafik *Weight Gradient Distribution Layer* 0 pada Model Tanpa Normalisasi



Gambar 2.2.1.5.4 Grafik *Weight Gradient Distribution Layer* 0 pada Model dengan RMSNorm

Pada distribusi gradien, kedua model sama-sama menunjukkan bahwa gradien terpusat di sekitar nol. Namun, pada model dengan RMSNorm, distribusi gradien terlihat sedikit lebih menyebar dibandingkan model tanpa normalisasi. Hal ini menunjukkan bahwa RMSNorm mempengaruhi dinamika *update* bobot dengan menghasilkan variasi gradien yang lebih besar, meskipun tetap berada dalam rentang yang stabil.

Berdasarkan hasil pengujian, penggunaan RMSNorm tidak memberikan peningkatan performa yang signifikan dibandingkan model tanpa normalisasi pada dataset ini. Hal ini menunjukkan bahwa efektivitas RMSNorm sangat bergantung pada konfigurasi model dan dataset yang digunakan. Pada pengujian ini, pengaruh RMSNorm lebih terlihat pada aspek stabilitas parameter dibandingkan peningkatan performa prediksi.

2.2.1.6 Uji Perbandingan *Library Sklearn*

Pada pengujian ini dilakukan perbandingan antara model Feedforward Neural Network (FFNN) yang diimplementasikan secara manual dengan model MLPClassifier dari *library sklearn*. Kedua model dilatih dengan arsitektur dan *hyperparameter* utama yang dibuat semirip mungkin, yaitu dua *hidden layer* berukuran 16 dan 8 neuron, fungsi aktivasi ReLU pada *hidden layer*, *learning rate* 0.01, *batch size* 32, tanpa regularisasi, serta jumlah *epoch* 500. Selain itu, pada model *sklearn* digunakan solver='sgd', momentum=0.0, dan early_stopping=False agar konfigurasi pelatihan lebih sebanding dengan model *custom* FFNN.

```
Custom FFNN
Output is truncated. View as a scrollable element or open in a text editor.
```

	precision	recall	f1-score	support
Not Placed	0.691643	0.624187	0.656186	769.0
Placed	0.778714	0.826158	0.801734	1231.0
macro avg	0.735178	0.725172	0.728960	2000.0
weighted avg	0.745235	0.748500	0.745771	2000.0

```
Accuracy  : 0.7485
Macro F1  : 0.7290
```

```
Sklearn MLPClassifier
```

	precision	recall	f1-score	support
Not Placed	0.683168	0.628088	0.654472	769.0
Placed	0.778809	0.818034	0.797940	1231.0
macro avg	0.730989	0.723061	0.726206	2000.0
weighted avg	0.742035	0.745000	0.742776	2000.0

```
Accuracy  : 0.7450
Macro F1  : 0.7262
```

Gambar 2.2.1.6.1 Hasil Evaluasi Model FFNN dan *sklearn*

	Model	Accuracy	Macro F1
0	Custom FFNN	0.7485	0.728960
1	Sklearn MLPClassifier	0.7450	0.726206

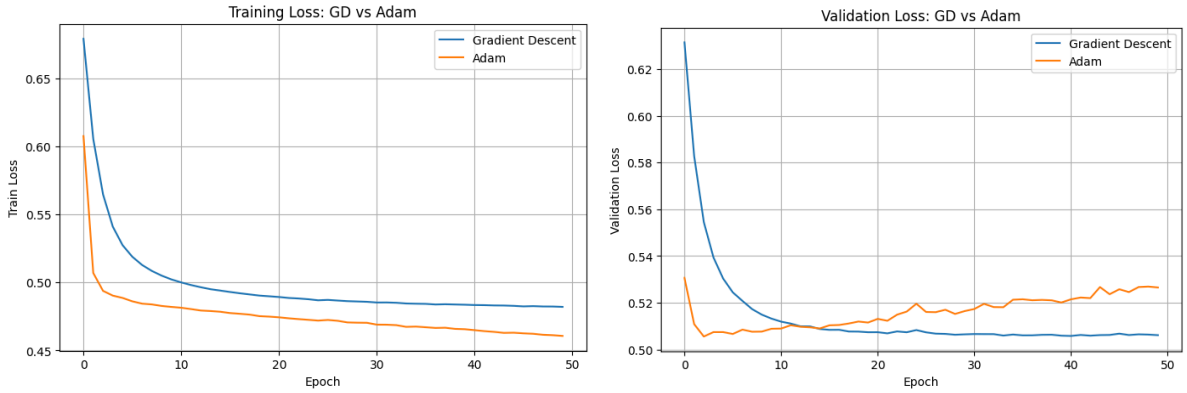
Gambar 2.2.1.6.2 Tabel Perbandingan Nilai *Accuracy* dan *Macro* F1 FFNN dan *sklearn*

Berdasarkan hasil pengujian, model *custom* FFNN memperoleh nilai *accuracy* sebesar 0.7485 dan *macro*-F1 sebesar 0.7290, sedangkan model *sklearn* MLPClassifier memperoleh nilai *accuracy* dan *macro*-F1 sebesar 0.7450 dan *macro* F1 sebesar 0.7262. Hasil ini menunjukkan bahwa performa kedua model sangat berdekatan, tetapi model FFNN yang diimplementasikan secara manual sedikit lebih unggul pada konfigurasi pengujian ini. Perbedaan tersebut tetap tergolong kecil, sehingga keduanya dapat dikatakan memberikan performa yang sebanding.

Kedekatan performa antara kedua model menunjukkan bahwa implementasi FFNN dari awal telah berjalan dengan baik dan mampu menghasilkan prediksi yang hampir mirip dengan MLPClassifier dari sklearn. Keunggulan kecil pada model kustom kemungkinan dipengaruhi oleh detail implementasi, skema inisialisasi, serta pelatihan yang tidak sepenuhnya sama dengan *library sklearn*, meskipun *hyperparameter* utamanya sudah disamakan. Dengan demikian, hasil ini menunjukkan bahwa model FFNN yang dibangun sudah benar secara fungsional dan cukup bagus untuk digunakan pada dataset ini.

2.2.1.7 Pengujian *Optimizer* Adaptive Moment Estimation (Adam)

Pada pengujian ini dilakukan analisis terhadap perbandingan performa antara *optimizer* Gradient Descent dan Adam. Adam merupakan *optimizer* yang menggabungkan momentum dan *adaptive learning rate* sehingga dapat mempercepat proses konvergensi dibandingkan metode Gradient Descent biasa. Pengujian ini dilakukan dengan menggunakan arsitektur model yang sama, dengan perbedaan hanya pada *optimizer* yang digunakan. Evaluasi dilakukan dengan membandingkan grafik *training loss*, *validation loss*, serta metrik performa akhir berupa *accuracy* dan *macro* F1-*score*.



Gambar 2.2.1.7.1 Grafik *Training Loss* dan *Validation Loss* pada GD vs Adam

Berdasarkan grafik *training loss*, terlihat bahwa model dengan *optimizer* Adam mengalami penurunan *loss* yang jauh lebih cepat dibandingkan Gradient Descent. Pada beberapa *epoch* awal, Adam sudah mencapai nilai *loss* yang rendah, sementara Gradient Descent membutuhkan lebih banyak *epoch* untuk mencapai kondisi yang sama. Hal ini menunjukkan bahwa Adam memiliki kecepatan konvergensi yang lebih tinggi.

Pada grafik *validation loss*, model dengan Adam juga menunjukkan penurunan yang cepat di awal. Namun, setelah beberapa *epoch*, terlihat bahwa *validation loss* pada Adam cenderung meningkat sedikit, yang mengindikasikan adanya kecenderungan *overfitting* lebih awal. Sebaliknya, Gradient Descent menunjukkan pola yang lebih stabil dengan perubahan *validation loss* yang lebih kecil.

	Model	Accuracy	Macro F1
0	Gradient Descent	0.752	0.730574
1	Adaptive Moment Estimation	0.743	0.717582

```
Epoch GD mencapai threshold: 9
Epoch Adam mencapai threshold: 2
```

Gambar 2.2.1.7.2 Perbandingan Hasil Prediksi GD dan Adam

Dari segi jumlah *epoch*, model dengan Adam mampu mencapai kondisi konvergen hanya dalam 2 *epoch*, sedangkan Gradient Descent membutuhkan 9 *epoch*. Hal ini memperkuat bahwa Adam lebih efisien dalam proses *training*. Namun, jika dilihat dari performa akhir, model dengan GD sedikit lebih unggul dengan nilai *accuracy* sebesar 0.752 dan *macro* F1-*score* sebesar 0.73, dibandingkan dengan Adam yang memperoleh *accuracy* sebesar 0.743 dan *macro* F1-*score* sebesar 0.717.

Hal ini menunjukkan bahwa meskipun Adam lebih cepat dalam proses konvergensi, GD cenderung menghasilkan generalisasi yang sedikit lebih baik pada pengujian ini. Maka, dapat disimpulkan bahwa Adam lebih unggul dalam hal efisiensi dan kecepatan konvergensi, sedangkan Gradient Descent lebih stabil dalam menghasilkan performa akhir yang sedikit lebih baik.

BAB 3 KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa model *FeedForward Neural Network* (FFNN) yang dibangun dari awal telah berhasil diimplementasikan dengan baik dan mampu melakukan proses klasifikasi pada dataset yang digunakan. Model ini telah mendukung berbagai komponen penting seperti fungsi aktivasi, fungsi *loss*, inisialisasi bobot, serta mekanisme *forward* dan *backward propagation*.

Dari hasil pengujian *hyperparameter*, ditemukan bahwa penambahan kompleksitas model seperti *depth* dan *width* tidak selalu meningkatkan performa. Model yang terlalu kompleks cenderung mengalami *overfitting*, sehingga diperlukan pemilihan arsitektur yang sesuai dengan karakteristik dataset. Selain itu, pemilihan *learning rate* juga berpengaruh terhadap kestabilan proses *training*.

Pada pengujian regularisasi, terlihat bahwa L1 dan L2 memberikan pengaruh terhadap distribusi bobot dan proses pembelajaran model. Regularisasi membantu mengontrol nilai bobot agar tidak terlalu besar, meskipun pada konfigurasi yang digunakan pengaruhnya terhadap peningkatan performa belum terlalu signifikan. Sementara itu, penggunaan RMSNorm membantu menjaga kestabilan nilai bobot dan gradien, tetapi tidak memberikan peningkatan performa yang signifikan pada dataset ini.

Hasil perbandingan dengan model MLPClassifier dari *library sklearn* menunjukkan bahwa performa model yang diimplementasikan secara manual sudah sebanding, bahkan sedikit lebih unggul. Hal ini menunjukkan bahwa implementasi yang dibuat sudah benar secara fungsional. Selain itu, pada pengujian *optimizer*, Adam terbukti lebih cepat dalam mencapai konvergensi, namun Gradient Descent menghasilkan performa akhir yang sedikit lebih baik.

3.2 Saran

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan lebih lanjut. Pertama, eksperimen dapat dikembangkan dengan melakukan tuning hyperparameter yang lebih luas, seperti variasi *learning rate*, jumlah *epoch*, serta kombinasi arsitektur jaringan yang lebih beragam.

Kedua, implementasi model dapat diperluas dengan menambahkan fitur lain seperti *optimizer* tambahan, teknik normalisasi lain, atau metode regularisasi yang lebih kompleks. Hal ini dapat membantu meningkatkan performa model serta memberikan perbandingan yang lebih komprehensif. Ketiga, evaluasi model dapat diperluas dengan menggunakan dataset lain atau melakukan *cross-validation* yang lebih mendalam agar hasil yang diperoleh lebih *general* dan tidak bergantung pada satu pembagian data saja.

PEMBAGIAN TUGAS

NIM	Nama	Pembagian Tugas
13523078	Anella Utari Gunadi	Implementasi fungsi aktivasi softmax, *class NeuralNetwork*, *debug data preprocessing*, pengujian regularisasi, implementasi dan pengujian normalisasi RMSNorm, perbandingan *library sklearn*, implementasi dan pengujian *optimizer* Adam, pengerjaan laporan.
13523079	Nayla Zahira	Implementasi fungsi aktivasi linear, sigmoid, leaky relu, elu, dan swish, implementasi class FFNN, implementasi fungsi loss cce, implementasi pengujian fungsi aktivasi dan learning rate, pengerjaan laporan.
13523080	Diyah Susan Nugrahani	Init struktur repository, implementasi fungsi aktivasi ReLU dan Hyperbolic Tangent, implementasi initializer, implementasi MSE, implementasi Activation Layer, pengujian *depth and width,* inisialisasi notebook EDA dan *preprocessing* awal, laporan, debugging.

REFERENSI

Kingma, D. P., & Ba, J. (2014). *Adam: A Method for Stochastic Optimization*. https://arxiv.org/pdf/1412.6980.

Scikit-learn. (2024). *MLPClassifier Documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifier.

LAMPIRAN

Lampiran Form Penggunaan AI

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Menggunakan Gemini (*Thinking* dan *Fast*) untuk diskusi struktur repository awal, debugging kode error, dan pengecekan struktur kalimat.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		